Introduction

In this lesson, we will talk about strictness of typing and see how to enable strict flags.

WE'LL COVER THE FOLLOWING

- Overview
- Enabling compiler flags

Overview

The TypeScript compiler is very configurable. There are tens of options that regulate different aspects of type checking and compilation. In this chapter, we're going to look at some of the compiler flags that let you decide how strict the compiler should be when enforcing type safety.

We use TypeScript because it helps us detect errors at compile time before they hit production. The stricter the checks TypeScript performs, the more errors it will be able to catch. Therefore, we should aim to make the checks as strict as possible. This can be achieved by setting the `strict` compiler flag to `true`. The `strict` flag is responsible for enabling a group of several flags. We're going to focus on the most important flags that will have the biggest impact on how you write code.

Enabling compiler flags

Enabling TypeScript compiler flags is very straightforward. Your project should contain a file called `tsconfig.json` on the top level. Inside the file, there should be a section called `compilerOptions`. Inside this section, you can add Boolean properties for every flag that we discuss in this course.

For example: to enable `strictNullChecks`, you should modify `tsconfig.json` to look like this:

```
{
  // some other settings

  "compilerOptions": {
    // some other options
    "strictNullChecks": true,
  }
}
```

If you're using an IDE such as Visual Studio Code or WebStorm, it should automatically pick up the changes you made in the file.

In the next lesson, we'll talk about how enabling the `noImplicitAny` flag increases strictness of type checking.

No Implicit `any`

This lesson introduces the `noImplicitAny` compiler flag.

WE'LL COVER THE FOLLOWING

- Definition
- `noImplicitAny` in practice

Definition

The first and the most important flag is `noImplicitAny`. Let's look at its definition from the documentation:

> Raise error on expressions and declarations with an implied any type.

In other words, when you enable the flag, TypeScript will throw an error whenever it's impossible to infer the type of something. Without the flag, TypeScript would automatically assume that the type is `any`. With the flag enabled, you have to provide type annotation. You can still use `any`, but now you have to do this explicitly.

`noImplicitAny` in practice

Let's look at the example of untyped function arguments. It's not possible for TypeScript to infer their types unless you provide them with default values; so if you don't type them explicitly, it will throw an error.

```
// ⚪ Parameter 'person' implicitly has an 'any' type.(7006)
function sayHello(person) {
  console.log(`Hello, ${person.name}`);
}
```

Run the code to see the compile error. `noImplicitAny` flag enabled.

Adding the type annotation fixes the error.

```
interface Person {
    name: string;
}

function sayHello(person: Person) {
  console.log(`Hello, ${person.name}`);
}
```

Run the code to verify there are no errors. `noImplicitAny` enabled.

Why is `noImplicitAny` a good thing? Having `person` typed as `any` essentially breaks type safety for the whole function. You cannot rely on TypeScript to check whether the reference to the `name` property is correct. While this example is trivial, imagine having a 100-line function; losing type-safety in such a huge chunk of code is a big deal.

Furthermore, `any` can propagate. In the example below, the return type of `getName` is inferred as `any`. This means that you can break type safety not only inside the function but also in the places it's being used.

```
function getName(person) {
  return person.name;
}
```

Hover over `getName` to see that it's type is inferred to `any`

In the next lesson, we'll discuss how to deal with type errors that materialize when `noImplicitAny` is enabled.

Fixing `noImplicitAny` Errors

This lesson discusses techniques of addressing compilation errors caused by enabling `noImplicitAny`.

WE'LL COVER THE FOLLOWING

- Explicit `any` annotation
- `TODO` aliasing technique
- Exercise

Explicit `any` annotation

The simplest way to fix errors caused by enabling `noImplicitAny` is simply explicitly marking the type as `any`. How is this better than skipping the type annotation altogether? It's still unsafe, right?

Correct; but with explicit annotations, it's much easier to spot these occurrences (for example, during code reviews). Because of the propagating aspect of `any`, it's easy to not realize that some portion of code is not type-checked. Making `any` explicit addresses this issue.

```
function sayHello(person: any) {
  console.log(`Hello, ${person.name}`);
}
```

Run the code to see that there are no compile errors thanks to explicit `any` annotation. `noImplicitAny` enabled.

`TODO` aliasing technique

There is a useful trick that can help you make `any` types stand out even more. It may happen that you need to introduce the `any` annotation because you don't have enough time or context to create a proper type for some variable

don't have enough time or context to create a proper type for some variable

or function parameter. In this case you can create an alias for `any` that starts with the `TODO` prefix.

```
type TODOPerson = any;

function sayHello(person: TODOPerson) {
  console.log(`Hello, ${person.name}`);
}
```

One of the advantages of creating this alias is that it's even easier to spot. Furthermore, by reusing the same alias in several places, you clearly indicate that the same type should be used in these places. Using `any` wouldn't convey this information.

Exercise

Identify places that would trigger an error with `noImplicitAny` flag enabled. Fix the code to avoid errors.

*Note that the system can only verify that your code compiles without error. To check your solution, click on **Show solution** and compare the code.*

```
class CounterComponent {
  count;

  constructor(private loggingService) {}

  increment(by) {
    this.loggingService.log(`Incrementing by ${by}`);
    this.count = this.count + by;
  }
}
```

In the next lesson, we'll talk about a type-safe alternative to `any`, the `unknown` type.

The `unknown` Type

This lesson explains the `unknown` type and explains how it is a better alternative to using `any`.

WE'LL COVER THE FOLLOWING

- Overview
- Type assignability
- Usage example for `unknown` type
- No explicit `any`
- Exercise

Overview

When trying to eliminate `any` from your codebase, it's useful to know about the `unknown` type. It is a safer alternative to `any`. Both `any` and `unknown` represent an unknown type. However, there is a key difference between these two:

- all types are assignable to the `any` type and the `any` type is assignable to any other type
- all types are assignable to the `unknown` type, but the `unknown` type is not assignable to any type

	assignable to	assignable from
any	all types	all types
unknown	no types	all types

Type assignability

What does it mean when a type is assignable to another type? We say that

type `A` is assignable to type `B` if you can use a value of type `A` in all places where a value of type `B` is required.
Here are some examples of situations when type assignability is checked:

- assigning a value to a variable
- passing a function argument
- returning a value from a function that has an explicit return type

As mentioned, the key difference between `any` and `unknown` is that `any` is assignable to any other type, but `unknown` is not assignable to any type. Let's look at some examples.

```
function foo1(bar: any) {
  const a: string = bar; // no error
  const b: number = bar; // no error
  const c: { name: string } = bar; // no error
}

function foo2(bar: unknown) {
  const a: string = bar; // Type 'unknown' is not assignable to type 'string'.(2322)
  const b: number = bar; // Type 'unknown' is not assignable to type 'number'.(2322)
  const c: { name: string } = bar; // Type 'unknown' is not assignable to type '{ name: s
}
```

Run the code to see the compile errors.

On the other hand, all types are assignable to `unknown`.

```
let x: unknown;

x = 123; // no error
x = 'abc';  // no error
x = true; // no error
```

Run the code to see that there are no errors.

Why is this difference so important? If you have a value of `unknown` type, you need to cast it to some other type before you can do anything useful with it. The consequence of this is that `unknown` doesn't *propagate* like `any` does, which is much safer.

Usage example for `unknown` type

When trying to get rid of the `any` instances from your codebase, you might run into situations where you actually have no way of knowing the type of some value. This is especially true when the value is "external", meaning it's returned by a backend endpoint or is deserialized from local storage. In such cases, it's a good idea to type such value as `unknown` instead of `any`.

```
interface Article {
    title: string;
    body: string;
}

fetch("http://example.com")
  .then(response => response.json())
  .then((body: unknown) => {
    const article = body as Article;
    // we need to cast, otherwise we'd get an error
    console.log(article.title);
  });
```

In this example, we use type assertion to tell TypeScript that we know the type of `body`. It's still not type-safe because we could be wrong, but at least it's explicit. The proper solution here would be to perform a runtime check to make sure that `body` is indeed an `Article`. We'll look into such a solution in the lesson dedicated to user-defined type guards.

No explicit `any`

With the `unknown` type, it's possible to completely avoid having `any` in your codebase. While there is no compiler flag to enforce the absence of `any`, you might consider using the `no-explicit-any` ESLint rule for this purpose. You can find the rule here.

ESLint is the code linter (a tool that analyzes code for errors, bugs, or stylistic inconsistencies) for JavaScript and TypeScript. TypeScript used to have a dedicated linter called TSLint, but it was deprecated in favor of ESLint. ESLint is used with the `eslint-plugin-typescript` plugin. I highly recommend looking into ESLint as it may help make your types even more strict.

Exercise

Write a runtime check that could be used in the above code to make sure that

`body` is indeed an `Article`.

```
function isArticle(body: unknown): boolean {
    return null;
}
```

In the following lesson, we'll start discussing the most important compiler flag related to strictness, the `strictNullChecks` flag.

Strict Null Checks

This lesson introduces the most important `strict` mode compiler flag - `strictNullChecks`. This flag can help you elevate the type safety of your code to a completely new level. It addresses the billion-dollar mistake of programming languages, null references.

WE'LL COVER THE FOLLOWING

- The billion-dollar mistake
- Types as sets
- `strictNullChecks` explained

The billion-dollar mistake

The expression comes from Thomas Hoare, a famous and influential computer scientist who introduced null references to ALGOL in 1965. Many years later he admitted that it was a mistake, as it resulted in *"innumerable errors, vulnerabilities, and system crashes, which have probably caused a billion dollars of pain and damage in the last forty years."*

What does it mean in the context of JavaScript and TypeScript?

```
Uncaught TypeError: Cannot read property 'foo' of undefined
```

Are you familiar with this error message? I bet you are. It occurs whenever you're trying to access a property or method of an object that you think is present, but turns out to not be there.

There are two ways of representing empty values in JavaScript - `null` and `undefined`. This kind of error can occur in both cases. `strictNullChecks` enables you to detect such mistakes at compile-time. It is an invaluable help and, when done properly, can lead to the complete eradication of this class of runtime bugs.

Types as sets

Types as sets

Let's break down the official definition of `strictNullChecks` from the documentation.

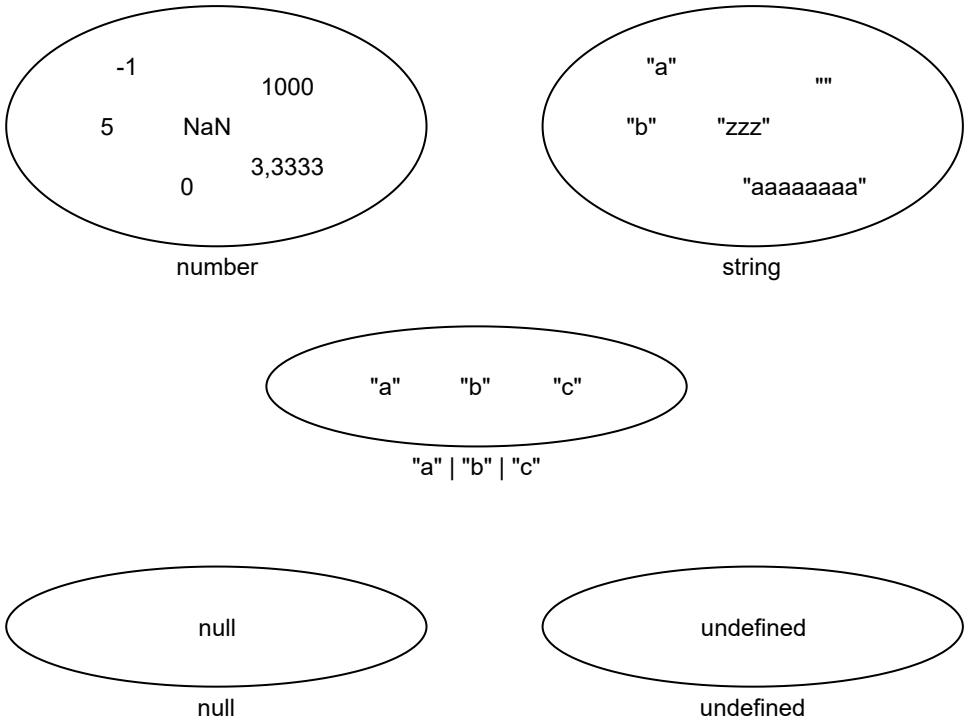
> In strict null checking mode, the null and undefined values are not in the domain of every type and are only assignable to themselves and any (the one exception being that undefined is also assignable to void).

The key sentence here is: *the null and undefined values are not in the domain of every type*. To fully understand this, let's look at TypeScript types from a mathematical perspective.

There is a concept called *set* in mathematics. A set is basically a grouping of things: numbers, fruit, other sets, functions. Sets can be finite or infinite. For example, `{1, 2, 5, 6}` is a finite set. The set of all the natural numbers is an infinite set.

Sets correspond to types very well. For example:

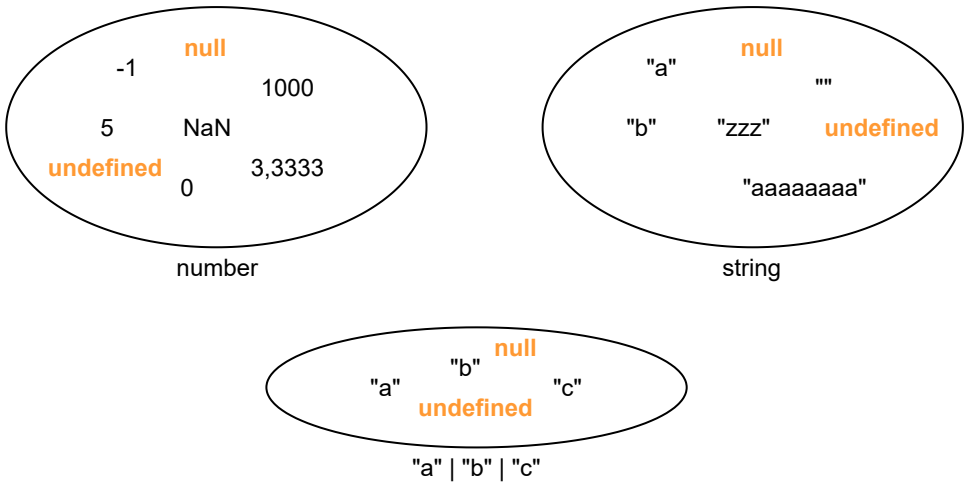
- `number` type can be looked at as a set of all possible numbers
- `string` type is a set of all possible strings
- `'a' | 'b' | 'c'` type corresponds to the `{'a', 'b', 'c'}` set
- `null` type is a singleton type with only one element, the `null` value (which happens to have the same name)
- `undefined` type is a singleton type with only one element, the `undefined` value (which happens to have the same name)



Set representation of some basic types

`strictNullChecks` explained

Back to `strictNullChecks`. The definition says that when the flag is *not* enabled, `null` and `undefined` values *are* in the domain of every type. In other words, `null` and `undefined` values belong to sets corresponding to every type.



Set representation of some basic types in TypeScript with `strictNullChecks` disabled

This is bad. This means that you can assign `null` and `undefined` values to a variable of every type because they are in the domain of every type. Consequently, you can later call a method on a variable whose value is `undefined` or `null`.

```
interface Person {
  hello(): void;

}

const n: number = undefined;
const f: string = null;
const person: Person = null;

person.hello(); // ⚪ Runtime Error!
```

`strictNullChecks` flag disabled

With `strictNullChecks` enabled, this is no longer the case. `null` and `undefined` do *not* automatically belong to all the types. Therefore, TypeScript won't let you use `null` or `undefined` when a type that doesn't include these values is required (line 8). You can fix this by extending the type to include `null` or `undefined` (line 10).

```
interface Person {
  hello(): void;
}

function foo(person: Person) {
  person.hello();
}
foo(null); // ⚪ Error!

function bar(person: Person | null) {
  person.hello(); // ⚪ Error!
  if (person !== null) {
    person.hello();
  }
}
bar(null); // OK
```

`strictNullChecks` flag enabled

However, if you do that, TypeScript won't let you call a method on such a variable (line 11)! And it's completely right to do so; the value of this variable can be `null`, so you shouldn't be calling any methods on it without first making sure that it is not empty.

Why is this not the default behavior, you might ask. Interestingly, this is how

Why is this not the default behavior, you might ask. Interestingly, this is how popular languages like Java, C++, or C# work; they don't have null checks

enabled by default. This is the mental model that most programmers are used to. TypeScript authors decided that enabling strict null checks could make the learning curve too steep which could negatively impact TypeScript adoption. Fortunately, they also made it possible to opt-out of the million-dollar mistake and make your code much safer.

In the next lesson, we'll see how enabling the `strictNullChecks` flag changes the way we work with optional types.

Optionality

This lesson discusses how TypeScript analyses optional values in `if` statements when the `strictNullChecks` flag is enabled.

WE'LL COVER THE FOLLOWING

- Defining optionality
- Checking optionality
- Exercise

Defining optionality

As you saw in the previous lesson, strict null checks force you to explicitly distinguish between values that can be `null` or `undefined` and those that cannot be. You already saw how to do this with a union type.

```
interface Person {
  name: string;
}

let nullableJohn: Person | null;
let maybeUndefinedBob: Person | undefined;
let ambiguoslyEmptyAlice: Person | null | undefined;
```

There is another way to express optionality for function parameters and object properties.

```
interface Person {}

function hello(person?: Person) {}

interface Employee {
  id: number;
  name: string;
  supervisorId?: number;
}
```

```
hello();
const employee: Employee = {
  id: 1,
  name: 'John',
};
```

Run the code to see that there are no compile errors even though `person` argument and `supervisorId` property are skipped (`strictNullChecks` flag enabled).

The above definitions are the same as the ones below with one subtle difference.

```
function hello(person: Person | undefined) {}

interface Employee {
  id: number;
  name: string;
  supervisorId: number | undefined;
}
```

With top definitions, you can skip the function argument/object property altogether while with the bottom ones you need to explicitly provide them, even when they are `undefined`.

```
interface Person {}

function hello(person: Person | undefined) {}

hello();
```

Run the code to see how this time skipping `person` is not allowed. Modify the code to get it to compile (`strictNullChecks` flag is enabled).

It sometimes makes sense to use the latter form, as it might better reflect your intention. It ensures that the caller doesn't forget about some property or function argument.

Checking optionality

As previously mentioned, before you can do anything with an optional value

As previously mentioned, before you can do anything with an optional value, you need to check if it's empty.

```
interface Person { hello(): void }

function sayHello(person: Person | undefined) {
  person.hello(); // ● Error!

  if (person !== undefined) {
    person.hello(); // OK
  }
}
```

Run the code to see how line 4 produces an error (`strictNullChecks` flag is enabled).

TypeScript's type checker is clever enough to analyze the condition of the `if` expression and deduce that the type of `person` inside the `if` statement's body is narrowed to just `Person`. This is an example of a *type guard*, a concept that we will discuss in subsequent lessons.

The following variant is a common pattern to quickly check optionality.

```
if (person) {
  person.hello();
}
```

Most of the time it's fine, however, you need to be careful when using it with primitive types.

```
function sayHello(name?: string) {
  if (name) {
    console.log(`Name: ${name.toLowerCase()}`);
  }
}

sayHello('');
```

Run the code to see that it doesn't print anything, even though one might expect it to print "Name:" (`strictNullChecks` flag enabled).

This code won't print anything even though the provided value is defined. `if (name)` will check whether `name` is *falsy*. An empty string is a falsy value, so

`(name)` will check whether `name` is *falsy*. An empty string is a falsy value, so

the condition will evaluate to `false`. It might be fine in this case, but in general, it's safer to explicitly compare with `null` or `undefined`.

Exercise

Change the definition of `getArticleAuthorName` so that the code compiles with `strictNullChecks` enabled.

```
interface Author {
    name: string;
}

interface Article {
    title: string;
    author?: Author;
}

function getArticleAuthorName(article: Article) {
    return article.author.name;
}
```

In the following lesson, we'll discuss a flag that is heavily related to `strictNullChecks`, the `strictPropertyInitialization` flag.

Strict Property Initialization

This lesson introduces another compiler flag, `strictPropertyInitialization`, which brings more type safety to code that uses classes.

WE'LL COVER THE FOLLOWING

- Definition
- Fixing the error
  - Constructor initialization
  - In-place initialization
  - Adjusting the type

Definition

The flag we're talking about is called `strictPropertyInitialization`. As usual, let's look at the definition from the documentation. I think it's rather self-explanatory.

> Ensure non-undefined class properties are initialized in the constructor. This option requires – strictNullChecks be enabled in order to take effect.

Enabling the flag will force you to always initialize class properties or make it explicit in the type annotation that the property can be `undefined`.

Fixing the error

In the piece below, the `header` property produces an error because it's initialized neither in-place nor in the constructor.

```
class HeaderComponent {
    // ⊗ Property 'header' has no initializer and is not definitely assigned in the constru
    header: string;
```

```
    render() {
        return `<h1>${this.header.toUpperCase()}</h1>`;

    }
}
```

Run the code to see the error caused by enabling `strictPropertyInitialization`.

There are three ways to get rid of this error. First, you can initialize the property in the constructor.

Constructor initialization

```
class HeaderComponent {
    constructor(private header: string) {}

    render() {
        return `<h1>${this.header.toUpperCase()}</h1>`;
    }
}
```

Run the code to verify that there are no compile errors. `strictPropertyInitialization` enabled.

In-place initialization

Second, you can initialize `header` in-place.

```
class HeaderComponent {
    header: string = 'hello';

    render() {
        return `<h1>${this.header.toUpperCase()}</h1>`;
    }
}
```

Run the code to verify that there are no compile errors. `strictPropertyInitialization` enabled.

Adjusting the type

Finally, you can explicitly mark `header` as possibly undefined. Note that this

will result in an error inside `render` because now you're trying to call a method on a possibly undefined object.

```
class HeaderComponent {
    header: string | undefined;

    render() {
        // ⚪ Error! Object is possibly `undefined`
        return `<h1>${this.header.toUpperCase()}</h1>`;
    }
}
```

Run the code to verify that line 6 results in an error. `strictPropertyInitialization` enabled.

The third option helps understand why this flag is related to `strictNullChecks`. Leaving a property uninitialized is essentially equal to lying about its type. We say that `header` is a `string`, but if we don't initialize it, it is `undefined` and not a `string`.

One could argue that this compiler behavior should be part of the `strictNullChecks` flag. However, strict property initialization checks could lead to cumbersome code. We'll see an example of this in the next lesson.

In the next lesson, we'll talk about an escape hatch for `strictNullChecks` and `strictPropertyInitialization` flags.

Non-Null Assertion Operator

This lesson introduces the non-null assertion operator and discuss when it is ok to use it.

WE'LL COVER THE FOLLOWING

- Overview
- Using with class properties
- Using when enabling `strictNullChecks` on large codebases

Overview

So far, we haven't talked about how to fix the kind of errors that are triggered by strict null checks. The reason I haven't mentioned a non-null assertion operator is that you shouldn't be using it unless you have a very good reason to do so.

Non-null assertion operators are simply a way of telling the compiler, *trust me, I'm absolutely sure that this optional value is in fact never empty*. Syntactically, you can express this by postfixing the optional value with a `!` character.

```
interface Person {
    hello(): void;
}

function sayHello(person: Person | undefined) {
    person!.hello(); // no errors!
}
```

Run the code to verify that there are no compile errors despite the code not being safe. `strictNullChecks` flag enabled.

While you might think that a value is always non-empty, you could be wrong.

Or worse, you could be correct at the moment, but it will not be the case in the future, because some other developer will call your function with a `null` or `undefined`.

Using with class properties

One possible valid use-case for non-null assertion operators is when you are working with classes and you have the `strictPropertyInitialization` flag enabled.

```
class CounterService {
    counter: number | undefined;

    increase() {
        if (this.counter !== undefined) {
            this.counter += 1;
            this.printCounter();
        }
    }

    private printCounter() {
        console.log(this.counter!.toLocaleString());
    }
}
```

Run the code to see that there are no errors. `strictNullChecks` enabled.

Here, we know that the `printCounter` is only called inside `increase`, where we know that `this.counter` is defined. If we wanted to get rid of this error properly, we would need to introduce an `if` statement.

```
class CounterService {
    counter: number | undefined;

    increase() {
        if (this.counter !== undefined) {
            this.counter += 1;
            this.printCounter();
        }
    }

    private printCounter() {
        if (this.counter !== undefined) {
            console.log(this.counter.toLocaleString());
        }
    }
```

```
}
```

Run the code to see that that there are no errors.

However, this check would always yield true, so one could argue that it adds unnecessary runtime overhead just to satisfy the type system. Therefore, the solution with a non-null assertion operator might be preferred. On the other hand, it could happen that later, another developer calls `printCounter` method in a different context and causes a runtime error.

In summary, you need to be extra careful when using the operator.

Using when enabling `strictNullChecks` on large codebases

Another situation in which it might make sense to use this operator is when you want to enable strict null checks on a large, existing codebase. Enabling the flag in such a case might result in hundreds or thousands of errors. It's a huge effort to go through them all at once. On the other hand, you cannot merge your changes to the master if the whole project doesn't compile successfully. In this case, the non-null assertion operator might come in handy, as it will allow you to quickly suppress the errors that you don't have time to fix properly.

The final lesson in this chapter is an exercise in strict typing.

Exercise in Strictness

The code below compiles with the `strict` flag set to `false`. Adjust it so that it compiles when `strict` is set to `true` as well.

After you have resolved all compiler errors, can you spot some other issues with the code? Can it still throw runtime errors?

*Note that the system can only verify that your code compiles without errors. To check your solution, click on **Show solution** and compare the code.*

```
interface Address {
  street: string;
  city?: string;
}

interface User {
  name: string;
  address: Address;
  meta: Record<string, string>;
}

interface SuperUser extends User {
  permissions: string[];
}

class UserRepository {
  private users: User[];

  init() {
    this.users = [
      // Do not change the data. Let's assume it comes from the backend.
      {
        name: "John",
        address: undefined,
        meta: { created: "2019/01/03" }
      },
      {
        name: "Anne",
        address: { street: "Warsaw" },
        meta: {
          created: "2019/01/05",
          modified: "2019/04/02"
        }
      }
    ];
```

```
  }

  getUser(id) {

    return this.users[id];
  }

  getCities() {
    return this.users
      .filter(user => user.address.city)
      .map(user => user.address.city);
  }

  forEachUser(action: (user: User) => void) {
    this.users.forEach(user => action(user));
  }
}

const userRepository = new UserRepository();

console.log(userRepository.getUser(1).address.city.toLowerCase());

console.log(
  userRepository
    .getCities()
    .map(city => city.toUpperCase())
    .join(", ")
);

console.log(new Date(userRepository.getUser(0).meta.modfified).getFullYear());
```

Introduction

This chapter dives deep into one of the most confusing topics in TypeScript - generics. We'll start by discussing *function types* to make sure that you have a solid understanding of them, and then we get into *generic functions*.

Next, we discuss *generic interfaces* from two perspectives, Object-Oriented Programming and Functional Programming. Interfaces are the bread and butter of TypeScript, so it's necessary to look at them from a couple different angles.

Finally, we look at a few more advanced topics. *Type argument constraints* are a way of setting up some expectations about the type argument of a generic type. The next lesson introduces the concept of *type constructors*, a theoretical foundation of generic types that will come in handy when discussing more advanced types. Finally, *type argument propagation* talks about an advanced language feature that helps when typing higher-order generic functions.

Function Types

This lesson covers a brief overview of function types to ensure you have a solid foundation before the introduction to generic functions.

WE'LL COVER THE FOLLOWING

- Functions as values
- Callbacks
- Higher-order functions

Functions as values

Every value in TypeScript has a type. Since JavaScript is a functional language, this means that functions are values too. Therefore, they also have types.

The easiest way to check the type of a function is to assign it to a constant and hover over the constant inside the code editor.

```
const add = (x: number, y: number) => x + y;
```

Hover over `add` to see the inferred type.

The type of `add` is inferred to be `(x: number, y: number) => number`. As you can see, it consists of a list of function parameters with their types and function return type, separated by an arrow symbol.

You can also define functions using the classic `function` syntax. From the type perspective, there is no difference between these two versions.

```
function sub(x: number, y: number) {
    return x - y;
}

const operation: (x: number, y: number) => number = sub;
```

Hover over `sub` to see the inferred type.

There is one other way to type functions in TypeScript using the `Function` type. However, using it is highly discouraged. By using it, you declare only that something is a function; you provide no information about its parameter types while the return type is provided.

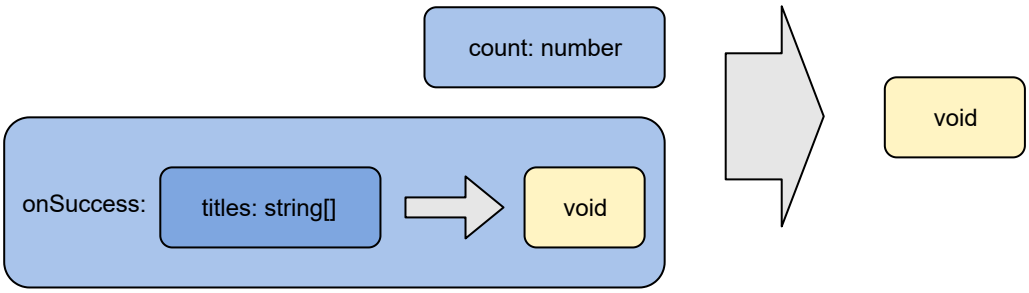
```
// Don't do this!
const operation2: Function = sub;
```

Callbacks

Callbacks are extremely popular in JavaScript and TypeScript. They are a great example of when a function type is necessary. A callback is essentially a function that you pass as an argument to another function. It is invoked when something specific happens, like a backend call returns or when some event occurs. It might be invoked with some arguments with data from the backend or an event object.

To express that a function accepts a callback as a parameter, you need to use a function type.

```
function fetchTitles(
    count: number,
    onSuccess: (titles: string[]) => void
) { /* ... */ }
```



Symbolic representation of the type of `fetchTitles`.

The `fetchTitles` function takes an `onSuccess` callback. The `onSuccess` callback is a function that should accept an array of strings and return nothing as a result. This type annotation carries a lot of information; it says that a list of fetched titles will be provided to this callback and that the return value of the callback will be ignored.

value of the callback will be ignored.

Higher-order functions

A function that accepts callbacks is an example of a *higher-order function*.

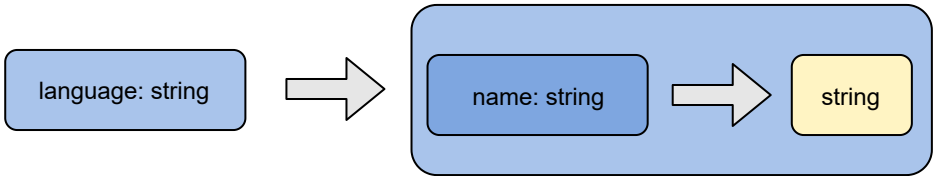
A higher-order function is such a function that:

- Has a parameter that has a function type
- OR, it's return type is a function type

Here's another example of a higher-order function that returns a function.

```
const getWelcomeFunction = (language: string) => {
    const hello = language === 'PL' ? 'Witaj' : 'Hello';
    return (name: string) => `${hello}, ${name}`;
}

const welcomeInPl = getWelcomeFunction('PL');
const welcomeInEn = getWelcomeFunction('EN');
```

Hover over `welcomeInPl` and `welcomeInEn` to see the inferred types.



Symbolic representation of the type of `getWelcomeFunction`.

The `getWelcomeFunction` takes a language code string and returns a function that will greet you in a specific language.

Higher-order functions are very common in JavaScript and TypeScript. Many generic functions are also higher-order functions, so it's extremely important to understand this concept.

In the next lesson, we'll look into generic functions.

Generic Functions

This lesson introduces generic functions.

WE'LL COVER THE FOLLOWING

- Motivation
- Let's use `any`!
- Generic functions
- Calling generic functions
- When should you use generic functions?

Motivation

Let's say you are adding types to some JavaScript codebase and you encounter this function:

```
function getNames(persons) {
  const results = [];
  for (let person of persons) {
    results.push(person.name);
  }
  return results;
}

console.log(getNames([
  { name: 'John' },
  { name: 'Alice' },
]));
```

Run the code to see what it does.

Typing this function is straightforward. It accepts an array of person objects as a parameter and returns an array of names (strings). For the person object, you can either create a `Person` interface or use one that you've already

created.

```
interface Person {
  name: string;
  age: number;
}

function getNames(persons: Person[]): string[] {
  /* ... */
}
```

Next, you notice that you don't actually need this function. Instead, you can use the built-in `Array.map` method.

```
interface Person { name: string; }

const persons: Person[] = [
  /* ... */
];

const names = persons.map(person => person.name);
```

Hover over `names` to see the inferred type.

Hmm, but what about types? You check the type of `names` and realize that it has been correctly inferred to `string[]`! How does TypeScript achieve this?

To properly understand this, let's try to type the following implementation of `map` function.

```
function map(items, mappingFunction) {
  const results = [];
  for (let item of items) {
    results.push(mappingFunction(item));
  }
  return results;
}

const persons = [
  { name: 'John' },
  { name: 'Alice' },
];
console.log(map(persons, person => person.name));
```

Run the code to see that it's working correctly.

The main issue with typing `map` is that you don't know anything about the type of the elements of the array it will be called with. What makes `map` so cool is that it works with *any* kind of array!

```
// Works with array of Persons
const names = map(persons, person => person.name);
// Works with array of names too
const uppercaseNames = map(names, name => name.toUpperCase());
// Works even with an array of numbers!
const evenNumbers = map([1, 2, 3, 4, 5], n => n * 2);
```

Let's use `any`!

As a first step, let's try using the `any` type to `map` this function.

```
function map(items: any[], mappingFunction: (item: any) => any): any[] {
  /* ... */
}
```

Let's break this down. `map` has two parameters. The type of the first one (`items`) is `any[]`. We tell the type system that we want `items` to be an array, but we don't care about the type of those items. The type of the second parameter (`mappingFunction`) is a function that takes `any` and returns `any`. Finally, the return type is again `any[]`; an array of *anything*.

Did we gain anything by doing this? We sure did! TypeScript now won't allow us to call `map` with nonsensical arguments:

```
interface Person { name: string; }
const persons: Person[] = [];

function map(items: any[], mappingFunction: (item: any) => any): any[] { return [];}

// ⚪ Error: 'hello' is not an array
map("hello", (person: Person) => person.name);
// ⚪ Error: 1000 is not a function
map(persons, 1000);
```

Run the code to see the errors.

Unfortunately, the types we provided are not precise enough. The purpose of TypeScript is to catch possible runtime errors as early as possible, like, at compile-time. However, the following calls won't give any compile errors.

```
interface Person { name: string; age: number;  }
const persons: Person[] = [{ name: 'John', age: 35 }];

function map(items: any[], mappingFunction: (item: any) => any): any[] { return items.map(map

// The second argument is a function that only works on numbers, not on `Person` objects.
// The result doesn't make sense.
console.log(map(persons, n => n + 5));
// We tell TypeScript that `ages` is an array of strings while in fact it will be an array of
// The second line results in a runtime error.
const ages: string[] = map(persons, person => person.age);
ages[0].toLowerCase();
```

Run the code to see the issues that were not captured by our definition of `map`.

How can we improve the typing of `map` so that the above examples would result in a compile-time error? Enter generics.

Generic functions

Making a function generic is (in this case) a way of saying "this function works with any kind of array" and while maintaining type safety at the same time.

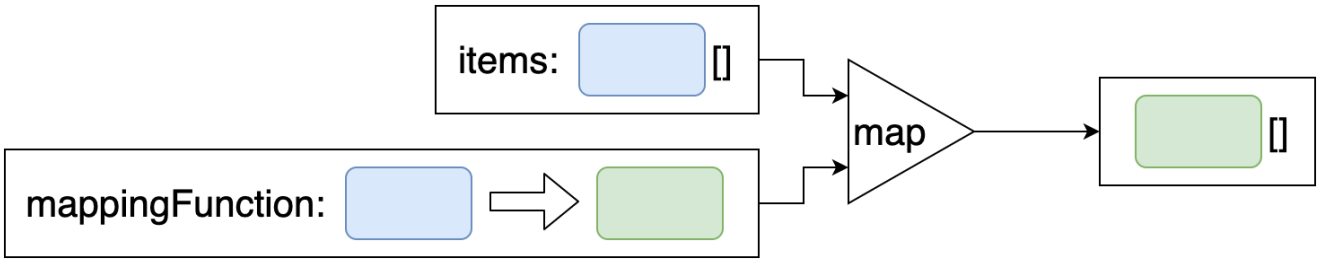
```
function map<TElement, TResult>(
  items: TElement[],
  mappingFunction: (item: TElement) => TResult
): TResult[] {
  /* ... */
}
```

We replaced `any` with `TElement` and `TResult` type parameters. Type parameters are *named* `any`. Typing `items` as `TElement[]` still means that it is an array of anything. However, because it's *named*, it lets us establish relationships between types of function parameters and the return type.

Here, we've just expressed the following relationships:

- `mappingFunction` takes anything as a parameter, but it must be *the same type of "anything"* as the type of elements in the `items` array
- `mappingFunction` can return anything, but whatever type it returns, it will be the same as the type of elements of the array returned by the `map` function

The picture below demonstrates these relationships. Shapes of the same color have to be of the same type.



You might have noticed the `<TElement, TResult>` that we added next to `map`. Type parameters have to be declared explicitly using this notation. Otherwise, TypeScript wouldn't know if `TElement` is a type argument or an actual type.

For some reason, it is a common convention to use single-character names for type parameters (with a strong preference for `T`). I'd recommend using full names, especially when you are not very experienced with generics. On the other hand, it's a good idea to prefix type arguments with `T` so that they're easily distinguishable from regular types.

Calling generic functions

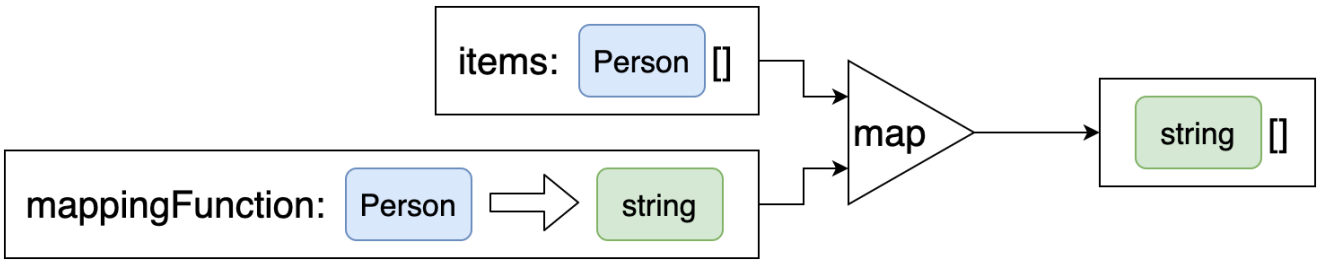
How to call a generic function? As we saw, generic functions have type parameters. These parameters are replaced with actual types when the function is called although technically, it's all happening at compile-time. You can provide the actual types using angle brackets notation.

```
interface Person { name: string; age: number;  }
const persons: Person[] = [{ name: 'John', age: 35 }];

function map<TElement, TResult>(
  items: TElement[],
  mappingFunction: (item: TElement) => TResult
): TResult[] {
```

```
  /* ... */
}



const names = map<Person, string>(persons, person => person.name);
```

Hover over `names` to see that type is inferred correctly.

Imagine that when we provide type arguments, `TElement` and `TResult` are replaced with `Person` and `string`.




```
function map<TElement, TResult>(
  items: TElement[],
  mappingFunction: (item: TElement) => TResult
): TResult[] {
  /* ... */
}

// ...becomes...

function map(
  items: Person[],
  mappingFunction: (item: Person) => string
): string[] {
  /* ... */
}
```

Having to provide type arguments when calling generic functions would be cumbersome. Fortunately, TypeScript can infer them by looking at the types of the arguments passed to the function. Therefore, we end up with the following code.

```
interface Person { name: string; age: number;  }
const persons: Person[] = [{ name: 'John', age: 35 }];

function map<TElement, TResult>(
  items: TElement[],
  mappingFunction: (item: TElement) => TResult
): TResult[] {
```

```
  /* ... */
}


const names = map(persons, person => person.name);
```

Hover over `map` on line 11 to see how its type arguments have been inferred correctly.

Woohoo! It looks exactly like the JavaScript version, except this one is type-safe! Contrary to the first version of `map`, the type of `names` is `string[]` instead of `any[]`. What's more, TypeScript is now capable of throwing a compile error for the following call.

```
interface Person { name: string; age: number;  }
const persons: Person[] = [{ name: 'John', age: 35 }];

function map<TElement, TResult>(
  items: TElement[],
  mappingFunction: (item: TElement) => TResult
): TResult[] {
  return [];
}

// ⚪ Error! Operator '+' cannot be applied to Person and 5.
map(persons, n => n + 5);
```

Run the code to see the error.

Here is a simplified sequence of steps that leads the compiler to throw an error.

1. The compiler looks at the type of `persons`. It sees `Person[]`.
2. According to the definition of `map`, the type of the first parameter is `TElement[]`. The compiler deduces that `TElement` is `Person`.
3. The compiler looks at the second parameter. It should be a function from `Person` to `TResult`. It doesn't know what `TResult` is yet.
4. It checks the body of the function provided as the second argument. It infers that the type of `n` is `Person`.
5. It notices that you're trying to add `5` to `n`, which is of type `Person`. This doesn't make sense, so it throws an error.

When should you use generic functions?

The good news is that, most likely, you won't be creating generic functions very often. It's much more common to call generic functions than to define them. However, it's still very useful to know how generic functions work, as it can help you understand compiler errors better.

As exemplified by `map`, functions that take arrays as parameters are often generic functions. If you look at the typings for the `lodash` library, you will see that nearly all of them are typed as generic functions. Such functions are only interested in the fact that the argument is an array, they don't care about the type of its elements.

In React framework, Higher-Order Components are generic functions, as they only care about the argument being a component. The type of the component's properties is not important.

In RxJs, most operators are generic functions. They care about the input being `Observable`, but they're not interested in the type of values being emitted by the observable.

The next lesson will walk you through an example of typing a non-trivial generic function.

Generic Functions Example - Typing `reduce`

This lesson walks through the implementation of an example generic function. Typing array utility functions is a great exercise that helps you fully understand the complexities of generics.

WE'LL COVER THE FOLLOWING

- What is `reduce`?
- Typing `reduce`
- Exercise

What is `reduce`?

`Array.reduce` is an extremely versatile function that lets you calculate results from all elements of an array. When calling `reduce` you specify a function that describes how to calculate the result for the next array element, given you already know the result for the previous array elements. It's like a functional version of a `foreach` loop with an additional variable that you change in the loop and return at the end.

```
const sum = [1, 2, 3, 4, 5].reduce((sum, el) => sum + el, 0);
console.log(sum);

interface Person {
    name: string;
    age: number;
}

const persons: Person[] = [
    { name: 'John', age: 30 },
    { name: 'Alice', age: 45 },
];

const ageByPerson = persons.reduce((result, person) => ({
    ...result,
    [person.name]: person.age
}), {});
console.log(ageByPerson);
```

Run the code to see how `reduce` works.

Our `reduce` will be a function that accepts three arguments:

- `array` - the array to be reduced
- `reducer` - a function that accepts an element of the array and the partial result (calculated based on elements processed so far)
- `initialResult` - a value that will be passed to the `reducer` before any array elements have been processed

Typing `reduce`

As you might have guessed, `reduce` is a generic, higher-order function. There will be two type arguments:

- `TElement` - the type of elements of the array
- `TResult` - the type of the calculated result

Let's look at the types of particular function arguments:

- `array` - since `TElement` is the type of array element, the type of the whole array is `TElement[]`
- `initialResult` - the type should be the same as the type of the final result; so it's `TResult`
- `reducer` - it's a function that takes a result (`TResult`) and an array element (`TElement`) and returns an updated result (`TResult`); therefore, it's type is `(result: TResult, el: TElement) => TResult`

Finally, the return type of the whole function is `TResult` (the type of the result we'd like to calculate).

Below you can find a simple implementation of `reduce` along with the usage examples with type arguments provided explicitly.

```
function reduce<TElement, TResult>(
    array: TElement[],
    reducer: (result: TResult, el: TElement) => TResult,
    initialResult: TResult
): TResult {
    let result = initialResult;
    for (const element of array) {
```

```
        result = reducer(result, element);
    }
    return result;

}

const total = reduce<number, number>([1, 2, 3, 4, 5], (sum, el) => sum + el, 0);

const ageByName = reduce<Person, Record<string, number>>(
    persons,
    (result, person) => ({
        ...result,
        [person.name]: person.age
    }),
    {}
);
```

Hover over `total` and `ageByName` to see whether their types are inferred correctly.

Exercise

Add types to the following `zip` function that takes two arrays and returns an array of pairs with elements from both arrays.

*Note that the system can only verify that your code compiles without errors. To check your solution, click on **Show solution** and compare the code.*

```
function zip(array1, array2) {
    const length = Math.min(array1.length, array2.length);
    const result = [];
    for (let i = 0; i < length; i++) {
        result.push([array1[i], array2[i]]);
    }
    return result;
}
```

The next lesson starts discussing generic interfaces.

Generic Interfaces - OOP Perspective

This lesson introduces generic types and discusses them from the perspective of Object-Oriented Programming.

WE'LL COVER THE FOLLOWING

- Interfaces in Object-Oriented Programming
- Generic OOP interfaces - `Observer`
- Generic OOP interfaces - `ReadonlyArray`

Interfaces in Object-Oriented Programming

Interfaces in TypeScript are very flexible and have multiple uses. One way of looking at them is from the perspective of Object-Oriented Programming (OOP). In this context, an interface defines a list of methods that a class needs to implement in order to conform to the interface.

OOP interfaces are often used to fulfill the **Dependency Inversion Principle** from SOLID design principles. A class (e.g., an Angular component) might have dependencies (e.g., an Angular service). Dependencies are things that you need to provide to the constructor of the class to create an instance of it. The principle says that it's much better to have interfaces than concrete classes as dependencies.

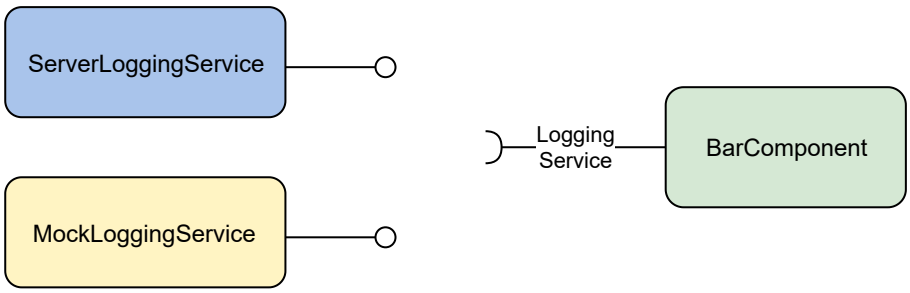
If the class requires a concrete class (e.g., a `ServerLoggingService` class), it's difficult to replace the dependency. For example, unit testing such a class is much harder since you have to provide an instance of this concrete class.

```
interface LoggingService {
    log(message: string): void;
}

class ServerLoggingService implements LoggingService {
    log(message: string): void {
        // send logs to the server
```

```
    }
}

class FooComponent {
    constructor(private loggingService: ServerLoggingService) {}
}

const foo = new FooComponent(new ServerLoggingService());
```

On the other hand, if the class only requires an interface (e.g., a `LoggingService` interface), you might provide any implementation of that interface; for example, a mock implementation for testing.

```
class BarComponent {
    constructor(private loggingService: LoggingService) {}
}

class MockLoggingService implements LoggingService {
    log(message: string): void {
        // don't do anything
    }
}

const bar = new BarComponent(new ServerLoggingService());
const testedBar = new BarComponent(new MockLoggingService());
```



In general, the OOP part of interfaces is their ability to express some *required behavior*, as opposed to *data* in the functional paradigm. This style of writing code might seem familiar to some Angular developers.

Generic OOP interfaces - `Observer`

An interface can specify a list of methods that must be provided by the implementation. A **generic interface** is very similar with the exception that some of the methods can be generic.

Let's look at the `Observer` interface from the RxJS library.

```
interface Observer<T> {
  closed?: boolean;
  next: (value: T) => void;
  error: (err: any) => void;
  complete: () => void;
}
```

The `Observer` interface represents an entity that can *observe* a stream of values. When the stream emits a value, the `next` method is called. When the stream encounters an error, the `error` method is called. Finally, when the stream completes, the `complete` method is called.

Stream of values is an abstract concept. We know that it represents some sequence of values, but we don't really care about the type of these values. That's why `Observer` is a generic interface. Its generic type argument, `T`, represents the type of values emitted by the stream. As mentioned previously, the `next` method is called for every emitted value. The value is passed as an argument to the `next`. We already know that `T` is the type of the value, so we can use it to type the `value` parameter of the `next` method.

Generic OOP interfaces - `ReadonlyArray`

Another interesting example is the `ReadonlyArray` interface that comes predefined with TypeScript.

```
interface ReadonlyArray<T> {
    // Skipped most of the methods for brevity.
    slice(start?: number, end?: number): T[];
    map<U>(callbackfn: (value: T, index: number, array: readonly T[]) => U
, thisArg?: any): U[];
    reduce<U>(callbackfn: (previousValue: U, currentValue: T, currentIndex
: number, array: readonly T[]) => U, initialValue: U): U;
    readonly [n: number]: T;
}
```

The `ReadonlyArray` is an interface that represents an immutable array. If you use `ReadonlyArray` to type the parameter of some function, you guarantee that the array passed to this function will never be modified. It's a compile-time guarantee, whereas at runtime, a regular array will be used.

guarantee, whereas at runtime, a regular array will be used.

`ReadonlyArray` achieves this by only specifying a non-mutating subset of regular array methods. For example, it doesn't specify `push` or `pop`. What's more, its indexer is marked as `readonly`.

The interface is generic for the same reason `Observer` was generic; we don't care about the type of values contained in the array. We mark the type of the values as `T` and can later use the types in the method declarations. For example, the `slice` method returns an array of `T` values where a slice of the array will have the same value type as the source array.

Interestingly, a generic interface can have actual generic methods, see `map` as an example. Feel free to compare it with the `map` function explained in previous lessons. Here, `map` takes only one type argument, `U`, which represents the type of elements of the resulting array. The type of `callbackFn` is a function type that accepts a `value` of type `T` (the type argument of the interface) and returns a `U` (the type argument of the method).

```
interface ReadonlyArray<T> {

    map<U>(
        callbackfn: (value: T) => U,
        thisArg?: any
    ): U[];

}
```

The next lesson looks at generic interfaces from a different angle.

Generic Interfaces - FP Perspective

This lesson talks about generic interfaces from the perspective of Functional Programming.

WE'LL COVER THE FOLLOWING

- Overview
- Working with generic data shapes

Overview

Unlike OOP, objects don't have behavior in Functional Programming. Interfaces are used not to enumerate methods of an object, but to describe the shape of the data contained by the object.

In this context, generic interfaces are used to describe a data shape when you don't know or care about the exact type of some properties of the interface. It often makes sense when the data types *contain* some value.

You can find an example of such an interface below. It represents a form field that could be part of some dynamic form implementation. We want to store certain information about a form field while we don't care about the type of data stored in that field. That's why we introduced the generic type parameter `T`.

```
interface FormField<T> {
    value?: T;
    defaultValue: T;
    isValid: boolean;
}
```

Working with generic data shapes

It's not possible to refer to a generic interface without providing its type argument. For example, if you want to write a function that accepts an

argument. For example, if you want to write a function that accepts an instance of a generic interface, you have two options:

- Write a specialized function that accepts a very specific instance of the interface (e.g. `FormField<string>`)
- Write a *generic* function

```
interface FormField<T> {
    value?: T;
    defaultValue: T;
    isValid: boolean;
}

// Very specialized. Only works with `FormField<string>`.
function getStringFieldValue(field: FormField<string>): string {
    if (!field.isValid || field.value === undefined) {
        // Thanks to the specialization, the compiler knows the exact type of `field.defaultV
        return field.defaultValue.toLowerCase();
    }
    return field.value;
}

// Generic. Can be called with any `FormField`.
function getFieldValue<T>(field: FormField<T>): T {
    if (!field.isValid || field.value === undefined) {
        // On the other hand, we don't know anything about the type of `field.defaultValue`.
        return field.defaultValue;
    }
    return field.value;
}
```

Hover over different variables to see how their types differ in both versions.

It's important to understand that in the second example (`getFieldValue`) `T` is the type argument **of the function**, not of the `FormField` interface. It gets passed only to `FormField` like any regular type does.

The next lesson talks about type argument constraints in generic types.

Type Argument Constraints

This lesson talks about generic type argument constraints.

WE'LL COVER THE FOLLOWING

- Overview
- Constraining with index type query operator
- Exercise

Overview

Sometimes it makes sense to put some restrictions on type arguments. Having these restrictions allows you to make assumptions about the type argument when referring to it inside some function.

You can express the restriction using the `extends` keyword. For example, you might decide to enforce the fact that `FormField` should only contain `string`, `number`, or `boolean` values.

You will notice that after introducing the restriction, the `getFieldValue` function no longer compiles.

```
interface FormField<T extends string | number | boolean> {
    value?: T;
    defaultValue: T;
    isValid: boolean;
}

// Type 'T' does not satisfy the constraint 'string | number | boolean'
function getFieldValue<T>(field: FormField<T>): T { /* ... */ }
```

Run the code to see the error.

The reason for this error is that the `T` type argument of the function has no restrictions. You're trying to pass it to `FormField` which only accepts types that extend `string`, `number` or `boolean`. Therefore, you get a compile error. To get rid of the error, you need to put the same or stricter restrictions on the type argument `T`.

```
interface FormField<T extends string | number | boolean> {
    value?: T;
    defaultValue: T;
    isValid: boolean;
}

// Type 'T' does not satisfy the constraint 'string | number | boolean'
function getFieldValue<T extends string | number>(field: FormField<T>): T {
    return field.value ?? field.defaultValue;
}
```

Run the code to verify that there are no more errors.

Constraining with index type query operator

A good example of using a type argument constraint is when typing a function that takes an object and a string representing the name of some property of this object. It's possible to type the function in such a way that if the provided string argument doesn't match any of the properties, the compiler will throw an error.

To achieve this, the function must take two type arguments: `T` and `K`. We can put a constraint on `K` and require it to extend `keyof T`. It's an example of using the *index type query operator* where `keyof K` returns a type that is a union of string literal types corresponding to the property names of `T`.

```
interface Person {
    name: string;
    age: number;
}

type PersonKeys = keyof Person; // 'name' | 'age'

function get<T, K extends keyof T>(object: T, key: K): T[K] {
    return object[key];
}

declare const person: Person;
get(person, 'name'); // ✅ No errors
```

```
get(person, 'name'); // ✅ No errors
get(person, 'foo'); // ⚫ Error!
```

Run the code to see the error. Hover over `PersonKeys` to see the inferred type.

Exercise

Add types to the following `pick` function that takes an array of items and a property name and returns an array of values of the provided property of array elements.

Note that the system can only verify that your code compiles without errors. To check your solution, click on ***Show solution*** *and compare the code.*

```
function pick(array, key) {
    const results = [];
    for (let element of array) {
        results.push(element[key]);
    }
    return results;
}
```

The next lesson talks about an important theoretical topic that is closely related to generic types.

Type Constructors

This lesson explains theoretical foundations of generic types and gives you a glimpse into category theory. It provides a mental model that will prove useful in the next chapters.

WE'LL COVER THE FOLLOWING

- Two worlds
- Category theory

Two worlds

Type constructor is a *function* (not a regular function, a mathematical function) that takes a type and returns a new type (based on the original type).

Notice that a generic interface with only one type argument is exactly that. For example, the `FormField` type from the previous lesson is a *function* with one type argument, `T`. It takes a type (e.g. `string`) and returns a new type (`FormField<string>` or `{ value?: string; defaultValue: string; isValid: boolean; }`).

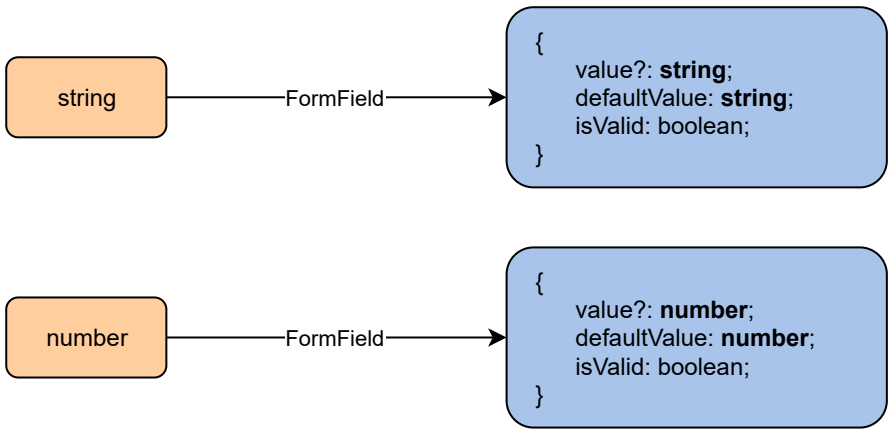


Diagram demonstrates how `FormField` acts like a function that maps a type to another type.

Try to think of *the world of types* as a universe that is parallel to *the world of values*. You're already familiar with *the world of values*, where functions take some values and return other values. In *the world of types*, type constructors

are functions that take some types and return other types.

Category theory

This analogy is not accidental. There is a branch of mathematics called **category theory**. It's a very abstract discipline that talks about **categories**. A category is anything that can be represented by a graph or a set of nodes with some arrows in between. It turns out that *the world of values* and *the world of types* are both categories! What's more, there are some universal laws that are true for all categories.

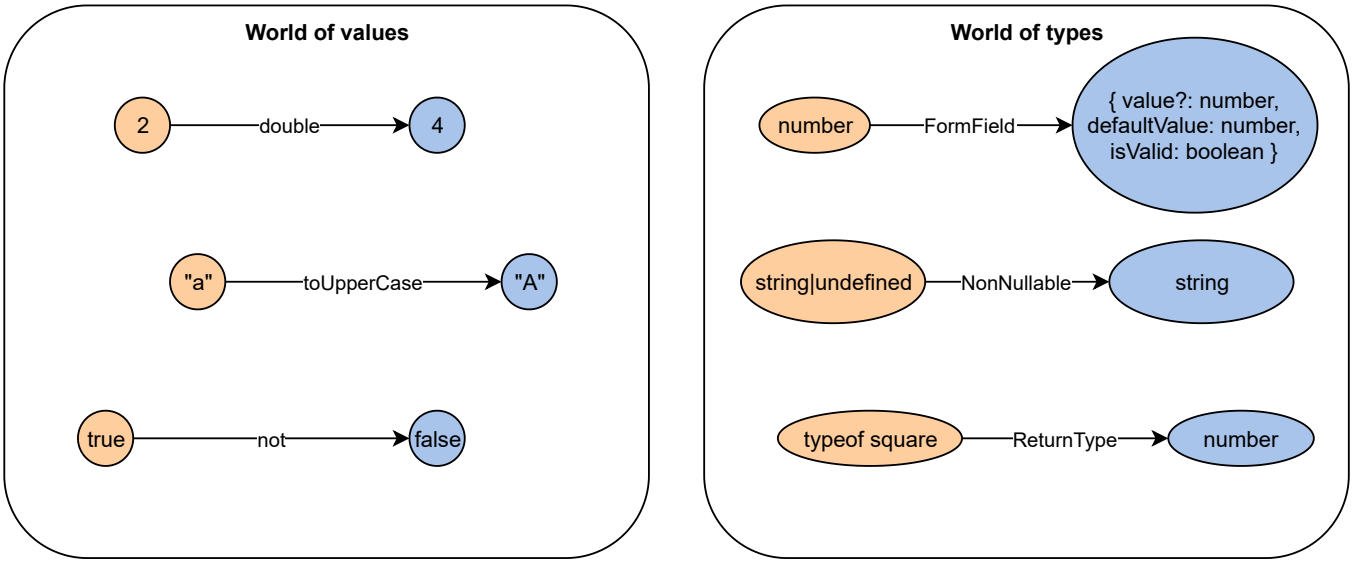


Diagram illustrates commonalities between the world of values and the world of types.

There are a few other interesting type constructors built into TypeScript. For example, `NonNullable<T>` is a *function* that takes a type and returns the same type with `null` and `undefined` removed from the domain.

`Partial` takes a type and returns a type with the same properties as the source, but each one optional.

`ReturnType` takes a function type and returns the return type of that function.

```
type X1 = NonNullable<string | undefined>; // X1 = string

type X2 = Partial<{ name: string; age: number; }>; // X2 = { name?: string; age?: number;}

function square(x: number): number { return x * x; }
type X3 = ReturnType<typeof square>; // X3 = number
```

Hover over `T1`, `T2` and `T3` to see the inferred types.

We'll look much closer at these types and more in the *Advanced types* chapter. The key takeaway from this lesson is that generic interfaces are very much like functions that transform types.

The final lesson in this chapter talks about an advanced typing scenario involving higher-order generic functions.

Type Argument Propagation

This lesson talks about an interesting language feature related to generic functions introduced in TypeScript 3.4.

WE'LL COVER THE FOLLOWING

- Overview
- Pointfree style
- Propagated generic type arguments
- Why does this matter?

Overview

Let's have a look at the following example. Imagine that you're fetching a collection of objects from some backend service and you need to map this collection to an array of identifiers.

```
interface Person {
  id: string;
  name: string;
  birthYear: number;
}

function getIds(persons: Person[]) {
    return persons.map(person => person.id);
}
```

Next, you decide to generalize the `getIds` function so that it works on any collection of objects that have the `id` property.

```
function getIds<T extends Record<'id', string>>(elements: T[]) {
  return elements.map(el => el.id);
}
```

Fair enough. However, the code for this simple function is quite verbose. Can

Fair enough. However, the code for this simple function is quite verbose. Can we make it more concise?

Pointfree style

We can do this by taking advantage of a functional programming technique called **pointfree style**. Ramda is a nice library that will let us compose this function from other functions: `map` and `prop`.

```
import * as R from 'ramda';

const getIds = R.map(R.prop('id'));
```

`map` is *partially applied* with a mapper function, `prop`, which extracts the `id` property from any object. The result of `getIds` is a function that accepts a collection of objects.

Sadly, TypeScript (pre 3.4) has bad news for you. The type of `getIds` is inferred to `(list: {}) => {}` which is not exactly what you'd expect.

You can explicitly type `map` but it makes the expression really verbose:

```
const getIds = R.map<Record<'id', string>, string>(R.prop('id'));
```

This is where *propagated generic type arguments* come in. In TypeScript 3.4 the type of `getIds` will correctly infer to `<T>(list: readonly Record<"id", T>[]) => T[]`. Success!

Propagated generic type arguments

Now that we know what *propagated generic type arguments* are about, let's decipher the name.

`R.map(R.prop('id'))` is an example of a situation when we pass a generic function as an argument to another generic function.

Before version 3.4 of TypeScript, the type of parameters of the inner function type was not *propagated* to the result type of the call.

Why does this matter?

Even if you're not particularly excited about pointfree style programming,

keep in mind that some popular libraries rely on function composition and partial application and will benefit from this change.

For example, in RxJS it is possible to compose new operators from existing ones using the `pipe` **function** (as opposed to the `pipe` method). TypeScript 3.4 will certainly improve typing in such scenarios.

Other examples include Redux (`compose` for middleware) and Reselect.

Introduction

This is the most important chapter of this course. It will teach you how to create new types from existing types.

First, we look at the most basic way of composing types with *union and intersection* operations. Union types are ubiquitous in TypeScript so it's important to thoroughly understand them.

Second, we introduce the topic of *discriminated unions*. It's a powerful mechanism that can be used to express business rules in the type system. Next, we'll talk about *state machines*, a very useful state management technique that can be implemented using discriminated unions. Finally, we'll compare discriminated unions and *subtyping* to see when to use which technique.

Union and Intersection Types

This lesson talks about the two most basic ways of composing types, union and intersection type operators.

WE'LL COVER THE FOLLOWING

- Overview
- Simple union types
- Union and intersection of object types
- Understanding union and intersection types
- Real-world example of intersection type

Overview

The primary way to compose types in TypeScript is via *union* and *intersection* type operations.

We've already seen union types in action in this course. I assume that you're more or less familiar with them, given that they're ubiquitous in TypeScript. In this lesson, I'd like you to gain an in-depth understanding of union types.

Have you ever wondered where these names come from? While you might have some intuition about what a union of two types is, the intersection is usually not understood well.

To properly understand this lesson, it's important that you're familiar with the idea of treating types as mathematical sets. Please read this section if you haven't yet.

Simple union types

Union type is very often used with either `null` or `undefined`.

```
const sayHello = (name: string | undefined) => { /* ... */ };
```

For example, the type of `name` here is `string | undefined` which means that either a `string` OR an `undefined` value can be passed to `sayHello`.

```
sayHello("milosz");
sayHello(undefined);
```

Looking at the example, you can intuit that a union of types `A` and `B` is a type that accepts both `A` and `B` values.

Union and intersection of object types

This intuition also works for complex types.

```
interface Foo {
    foo: string;
    xyz: string;
}

interface Bar {
    bar: string;
    xyz: string;
}

const sayHello = (obj: Foo | Bar) => { /* ... */ };

sayHello({ foo: "foo", xyz: "xyz" });
sayHello({ bar: "bar", xyz: "xyz" });
```

Run the code to see that there are not compile errors.

`Foo | Bar` is a type that has either all required properties of `Foo` OR all required properties of `Bar`. Inside the `sayHello` function, it's only possible to access `obj.xyz` because it's the only property that is included in both types.

What about the intersection of `Foo` and `Bar`, though?

```
const sayHello = (obj: Foo & Bar) => { /* ... */ };

sayHello({ foo: "foo", bar: "bar", xyz: "xyz" });
```

Now `sayHello` requires the argument to have both `foo` AND `bar` properties.

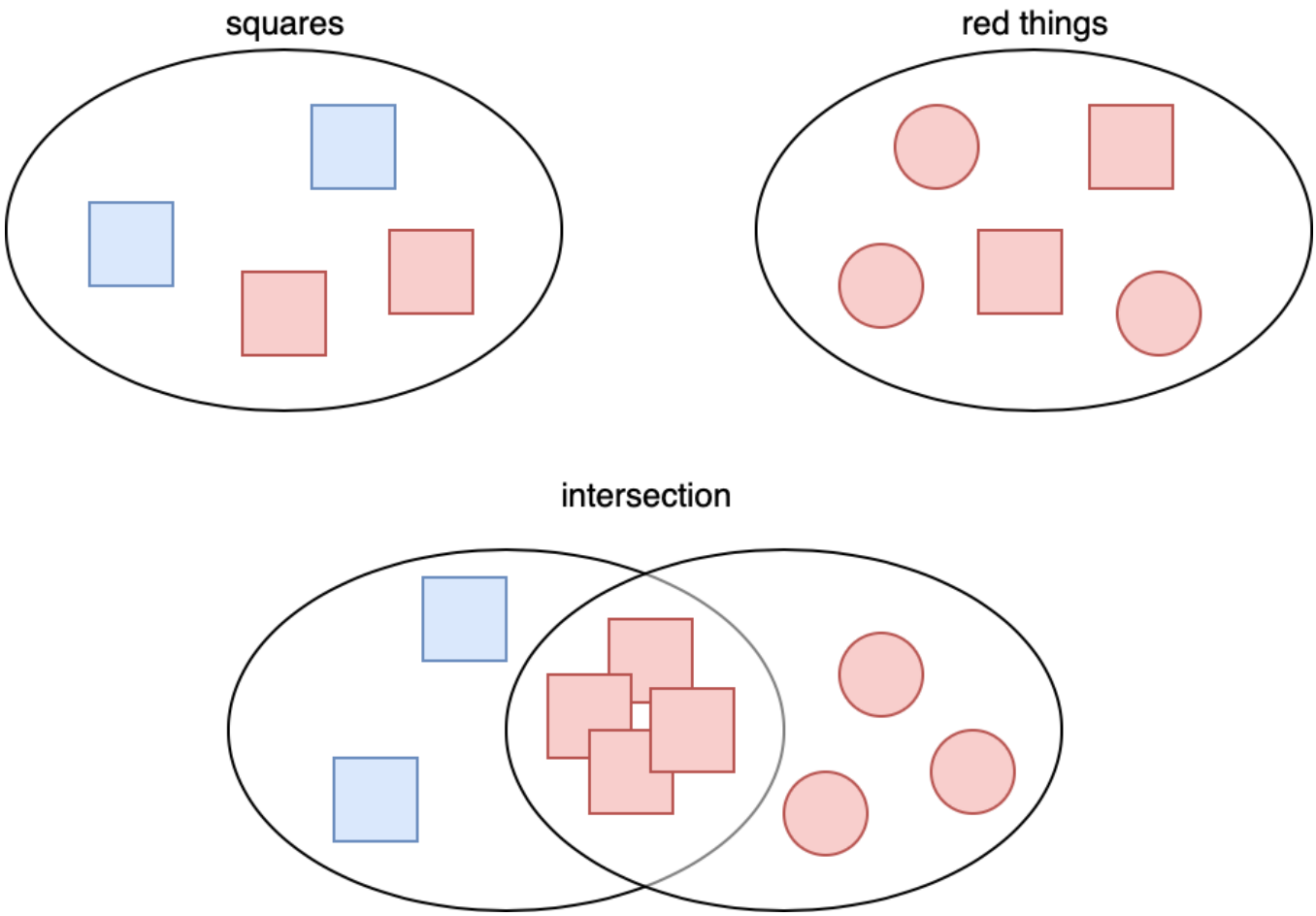
Inside `sayHello` it's possible to access `obj.foo`, `obj.bar`, and `obj.xyz`.

Hmm, but what does it have to do with *intersection*? One could argue that since `obj` has properties of both `Foo` and `Bar`, it sounds more like a union of properties and not an intersection. Similarly, a union of two object types gives you a type that only has the intersection of properties of constituent types.

It sounds confusing. I even stumbled upon a GitHub issue in the TypeScript repository ranting about the naming of these types. To understand the naming better we need to look at types from a different perspective.

Understanding union and intersection types

The confusion disappears if you look at union and intersection types from the perspective of set theory. In set theory, union and intersection are names of operations on sets. The union of two sets is a set that contains all elements from both sets. The intersection of two sets is a set that only contains elements that are present in both sets.



Armed with this knowledge, you're ready to understand the meaning of union and intersection types.

Union type `A | B` represents a set that is a union of the set of values associated with type `A` and the set of values associated with type `B`.

Intersection type `A & B` represents a set that is an intersection of the set of values associated with type `A` and the set of values associated with type `B`.

Therefore, `Foo | Bar` represents **a union** of the set of objects having `foo` and `xyz` properties and the set of objects having `bar` and `xyz`. Objects belonging to such sets all have the `xyz` property. Some of them have `foo` property and others have `bar` property.

`Foo & Bar` represents **an intersection** of the set of objects having `foo` and `xyz` properties and the set of objects having `bar` and `xyz`. In other words, the set contains objects that belong to the sets represented by both `Foo` and `Bar`. Only objects that have all three properties (`foo`, `bar` and `xyz`) belong to the intersection.

Real-world example of intersection type

Union types are quite widespread so let's focus on an example of an intersection type.

In React, when you declare a class component, you can parameterize it with the type of its properties:

```
class Counter extends Component<CounterProps> { /* ... */ }
```

Inside the class, you can access the properties via `this.props`. However, the type of `this.props` is not simply `CounterProps`, but:

```
Readonly<CounterProps> & Readonly<{ children?: ReactNode; }>
```

The reason for this is that React components can accept children elements:

```
<Counter><span>Hello</span></Counter>
```

The children element trees are accessible to the component via the `children` prop. The type of `this.props` reflects that. It's an intersection of (read-only) `CounterProps` and a (read-only) object type with an optional `children` property.

In terms of sets, it's an intersection of the set of objects that have properties as

In terms of sets, it's an intersection of the set of objects that have properties as defined in `CounterProps` and the set of objects that have the optional `children` property. The result is a set of objects that have all the properties of `CounterProps` and the optional `children` property.

The next lesson introduces discriminated union types, a very powerful special case of union types.

Discriminated Union Types

This lesson introduces the concept of discriminated union types.

WE'LL COVER THE FOLLOWING

- Overview
- Representing choice with optional properties
- Discriminated unions to the rescue
- Note on formatting

Overview

Now that you have a full understanding of union types, let's look at one of their special categories.

Creating **discriminated unions** is an extremely powerful way of composing types. Thanks to this mechanism you can write code that is type-safe beyond your imagination. Discriminated unions let you enforce some business logic rules **at compile-time**.

Representing choice with optional properties

Let's start by looking at an example. Imagine implementing an e-commerce application. When the customer registers on the app, they need to provide some contact details. One of the acceptance criteria says:

> Customer needs to provide either email or phone number.

The most straightforward way of defining the `Customer` type that satisfies this requirement is by using two optional properties.

```
interface Customer {
```

```
  name: string;
  email?: string;
  phone?: number;
}
```

This solution has one major drawback. It's possible to create a `Customer` with neither `email` nor `phone` provided. In other words, it's possible to create a `Customer` object that violates the business requirement.

You might argue that the type will be accompanied by a validation function that will throw an exception if the object doesn't have any of these two properties defined. This is slightly better, but still far from ideal because of the following:

- You will learn that the object is invalid at runtime
- When using `strictNullChecks`, you'll need to write `if` statements every time you want to access `email` or `phone`
- You might forget to call the validation function

Discriminated unions to the rescue

Instead of having two optional properties in `Customer` type, let's create a new `Contact` type that is a union of two interfaces. One union member has a non-optional `email` property and the other has a non-optional `phone` property.

These two union members follow a special convention. They both have a string literal property called `kind`. The type of that property is different in both members. This property is called a **discriminator**. It encodes type information into the object so that it is available at runtime. Thanks to this, TypeScript can figure out which union member you're dealing with by looking at the `kind` property.

```
interface EmailContact {
    kind: 'email';
    email: string;
}

interface PhoneContact {
    kind: 'phone';
    phone: number;
}

type Contact = EmailContact | PhoneContact;
```

```
interface Customer {
    name: string;
    contact: Contact;
}

function printCustomerContact({ contact }: Customer) {
    if (contact.kind === 'email') {
        // Type of `contact` is `EmailContact`!
        console.log(contact.email);
    } else {
        // Type of `contact` is `PhoneContact`!
        console.log(contact.phone);
    }
}
```

Hover over `contact` inside both branches of the `if` statement to see how the type is narrowed.

Here comes the power of discriminated unions. TypeScript analyses the code and sees an `if` statement that checks whether `contact.kind` is equal to `"email"`. If that's true, it can be certain that the type of `contact` is `EmailContact`, so it narrows the type inside the first `if` branch. The only other option for `contact` is to be a `PhoneContact`, so the type is narrowed to `PhoneContact` in the second `if` branch.

You might be worried about the usage of a *magic string* in the `if` statement. In fact, the string is not really *magic*. The type of `contact.kind` (outside of the `if`) is `'email' | 'phone'`. Try it yourself and see how the code editor suggests that the only possible values for `contact.kind` are `'email'` or `'phone'`.

We've made an illegal state unrepresentable. In the previous solution, it was possible to create a customer without phone or email (i.e., it was possible to create an illegal piece of application state). Now it's forbidden at compile time!

Note on formatting

`Contact` type can be written in a much terser way as it's not necessary to create standalone `PhoneContact` and `EmailContact` types - they can be inlined.

```
type Contact =
    | { kind: 'email', email: string }
    | { kind: 'phone', phone: number };
```

The first `|` has been added only for formatting purposes. It's allowed by TypeScript syntax and is a common convention to use when defining multi-

line unions.

The next lesson dives deeper into using discriminated unions.

Discriminated Unions in Practice

This lesson contains some facts about discriminated unions that may come in handy when using them in the wild.

WE'LL COVER THE FOLLOWING

- Non-string discriminators
- Multiple discriminating properties
- Algebraic data types
- Excercise: Precise domain modeling

Non-string discriminators

In the example from the previous lesson, we used a string literal property called `type` as a discriminator. First, the discriminator doesn't have to be called `"type"`. Any property name will be fine.

Second, the discriminator doesn't have to be a string. You can also use number literals, Boolean literals, or even enums!

The built-in `IteratorResult` type in TypeScript (starting from version 3.6) uses the Boolean property `done` as a discriminator.

```
type IteratorResult<T, TReturn = any> = IteratorYieldResult<T> | IteratorR
eturnResult<TReturn>;

interface IteratorYieldResult<TYield> {
    done?: false;
    value: TYield;
}

interface IteratorReturnResult<TReturn> {
    done: true;
    value: TReturn;
}
```

Using enums can help you avoid *magic strings* if that's an issue for you. One advantage of this approach is that renaming discriminating values becomes much easier.

```
const enum ContactType { Phone, Email };

type Contact =
    | { type: ContactType.Email, email: string }
    | { type: ContactType.Phone, phone: number };
```

Multiple discriminating properties

Interestingly, a discriminator does not have to be a single property. A group of literal properties can also act as a discriminator! In such a case, every combination of values marks a different member of the union type.

```
type Foo =
  | { kind: 'A', type: 'X', abc: string }
  | { kind: 'A', type: 'Y', xyz: string }
  | { kind: 'B', type: 'X', rty: string }

declare const foo: Foo;

if (foo.kind === 'A' && foo.type === 'X') {
  console.log(foo.abc);
}
```

Hover over `foo` inside the `if` statement to see how its type is narrowed.

Algebraic data types

Discriminated unions are not unique to TypeScript. Statically-typed functional languages often make heavy use of *algebraic data types*. You can find an example from Haskell compared with a TypeScript equivalent below.

```
data Tree = Empty
            | Leaf Int
            | Node Tree Tree
```

```
type Tree =
      | { type: 'empty' }
      | { type: 'leaf', value: number }
```

```
    | { type: 'node', left: Tree, right: Tree };
```

One might argue that discriminated unions do not feel natural because of the necessity of adding the extra property. Traditional statically typed languages (like Haskell) encode type information in runtime values. However, since TypeScript compiles to JavaScript, all the type information is lost at runtime. You need to add the information yourself, hence the extra property. TypeScript just happens to understand this convention and utilizes it when analyzing your code.

Excercise: Precise domain modeling

So far, we have seen an example of how discriminated unions help encode business rules in the type system. In this exercise, you're going to write types that capture a slightly more complex domain.

Create types for the following domain:

> Users place orders for products. Users have contact information, email or postal addresses, and at least one is required. Orders should include price, product name, quantity, payment date, paid amount, sending date, and delivery date.

*Note that the system can only verify that your code compiles without errors. To check your solution, click on **Show solution** and compare the code.*

```
type Order = {
  productName: string;
  price: number;
  quantity: number;
};
```

The next lesson talks about how discriminated unions can be leveraged to build state machines in TypeScript.

State Machines

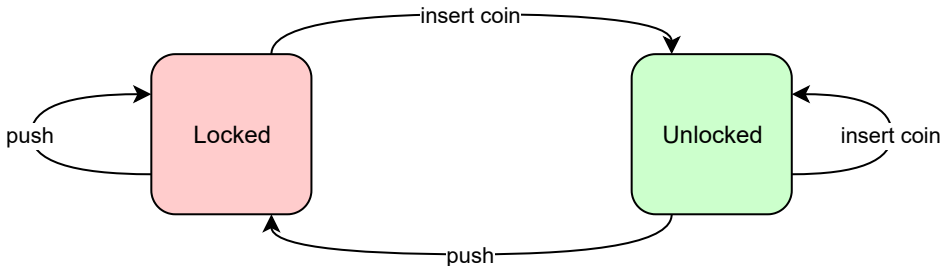
This lesson talks about state machines, a concept closely related to discriminated unions.

WE'LL COVER THE FOLLOWING

- Introduction to state machines
- Example: Data fetching workflow
- Exercise

Introduction to state machines

Managing the state is one of the hardest parts of frontend development. Many frameworks and libraries have been created to help with this task. One of the proposed approaches is to utilize **state machines**. A state machine is a directed graph where nodes represent all possible states of a view or of the whole app, and where edges represent possible transitions between the states.



Classic example of a state machine representing a turnstile. Turnstile has two states: locked and unlocked. You can unlock it by inserting a coin. If you push it, it's locked again. Pushing a locked turnstile doesn't affect the state. So doesn't inserting a coin into an unlocked turnstile.

An arrow between two nodes means that it's possible to go through one state to another via some action. All non-listed transitions are not possible. There can be a meaningful transition from a state to the same state that is marked by a circular arrow.

Using state machines is an explicit and safe way of dealing with the state. They're particularly useful when the state is very complex and involves

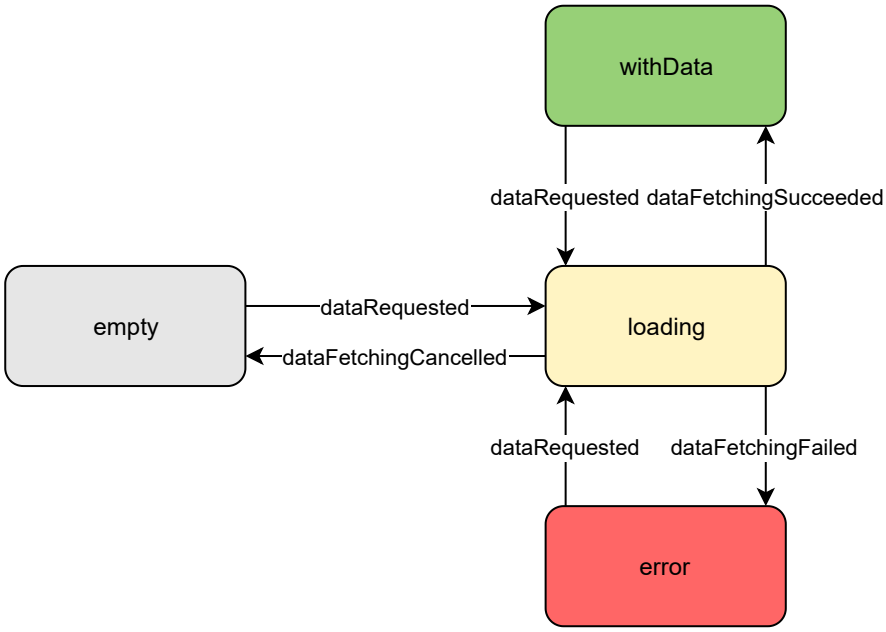
multiple transitions.

Example: Data fetching workflow

One very common task in web applications is loading data from the backend. Let's build a state machine that represents this workflow. The solution will be based on Redux architecture.

Let's say that the user can request data to be fetched by clicking on a button. It should also be possible to cancel the request. When data is being fetched, a loading indicator should be shown. Data fetching can be successful, in which case it will show the fetched data. It can also fail, in which case it will show an error.

Based on the above description, we can build the following state machine.



There are four states:

- *empty* - fetching has not started yet or has been canceled
- *loading* - fetching is in progress
- *withData* - fetching was successful
- *error* - fetching failed

Let's create a discriminated union that represents all of the possible states. Next, let's look at all possible actions that can result in state transitions. Finally, we'll build a reducer which describes all of the possible transitions.

```
type State =
    | { type: 'empty' }
    | { type: 'loading' }

    | { type: 'withData', data: string[] }
    | { type: 'error', errorMessage: string };

type Action =
    | { type: 'dataRequested' }
    | { type: 'dataFetchingSucceeded', data: string[] }
    | { type: 'dataFetchingFailed', errorMessage: string }
    | { type: 'dataFetchingCancelled' };

function reducer(prevState: State, action: Action): State {
    switch (prevState.type) {
        case 'loading':
            switch (action.type) {
                case 'dataFetchingSucceeded':
                    return { type: 'withData', data: action.data };
                case 'dataFetchingCancelled':
                    return { type: 'empty' };
                case 'dataFetchingFailed':
                    return { type: 'error', errorMessage: action.errorMessage };
                case 'dataRequested':
                    return prevState;
            }
        case 'empty':
        case 'error':
        case 'withData':
            switch (action.type) {
                case 'dataRequested':
                    return { type: 'loading' };
                case 'dataFetchingCancelled':
                case 'dataFetchingFailed':
                case 'dataFetchingSucceeded':
                    return prevState;
            }
    }
}
```

As you can see, the reducer closely matches the state machine. First, we switch over possible states. For each state, we defined what happens when a certain action is triggered. For some actions, we return `prevState` which means that this transition is not possible.

Note that we never defined the `default` case. Because of this, if we forgot to handle some state or transition, TypeScript would throw a compilation error. This pattern only works if you explicitly defined the return type of the containing function. It's extremely useful, even outside of the context of state machines.

As you can see, data fetching workflow is very clear when represented using a state machine. It's a type-safe approach that helps you break down the complexity in a very structured way.

Exercise

Implement the turnstile state machine.

```
type State =
    | { type: 'empty' }
    | { type: 'loading' }
    | { type: 'dataPartiallyLoaded', data: string[] }
    | { type: 'withData', data: string[] }
    | { type: 'error', errorMessage: string };

type Action =
    | { type: 'dataRequested' }
    | { type: 'dataFetchingInProgress', data: string[] }
    | { type: 'dataFetchingSucceeded', data: string[] }
    | { type: 'dataFetchingFailed', errorMessage: string }
    | { type: 'dataFetchingCancelled' };

function reducer(prevState: State, action: Action): State {
  return prevState;
}
```

The final lesson in this chapter compares discriminated unions with an equivalent OOP technique called subtyping.

Discriminated Unions vs Subtyping

This lesson discusses the differences between the two approaches of expressing choice in the type system.

WE'LL COVER THE FOLLOWING

- Overview
- Defining types
- Adding an operation
- Comparison

Overview

Subtyping is a form of type polymorphism that is very popular in Object-Oriented Programming. If you're familiar with OOP languages such as Java or C#, then you very likely know it already.

Subtyping offers an alternative to Algebraic Data Types (discriminated unions). In this lesson, I'd like to compare these two approaches.

Defining types

Let's go back to the example of a discriminated union representing a binary tree. It could, for example, be a tree of product categories in an e-commerce web application.

```
type Tree =
      | { type: 'empty' }
      | { type: 'leaf', value: number }
      | { type: 'node', left: Tree, right: Tree };
```

A tree can either be:

- a node with references to left and right subtrees
- a leaf containing a numerical value

- a leaf containing a numerical value
- empty

We can represent the same data structure using the OOP approach. We define an abstract class `Tree` and three concrete classes extending from `Tree`.

```
abstract class Tree {}

class Empty extends Tree { }

class Leaf extends Tree {
  constructor(private value: number) {
    super();
  }
}

class Node extends Tree {
  constructor(
    private left: Tree,
    private right: Tree,
  ) {
    super();
  }
}
```

Adding an operation

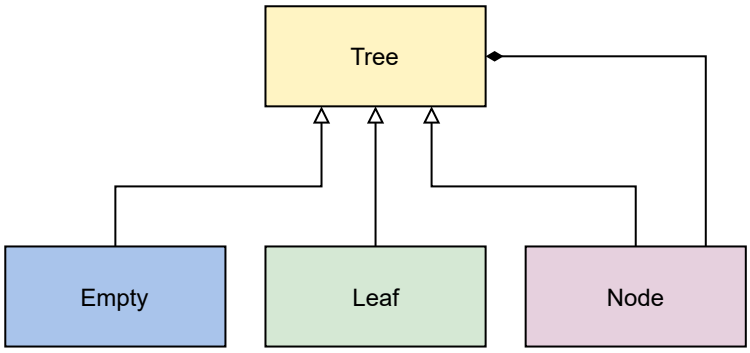
Now, let's say that we want to create a textual representation of the tree. Let's see how the implementation would look in both approaches.

```
type Tree =
      | { type: 'empty' }
      | { type: 'leaf', value: number }
      | { type: 'node', left: Tree, right: Tree };

function toString(tree: Tree): string {
    switch (tree.type) {
        case 'empty':
            return 'Empty';
        case 'leaf':
            return `Leaf(${tree.value})`;
        case 'node':
            return `Node(${toString(tree.left)}, ${toString(tree.right)})`;
    }
}
```

Hover over `tree` inside `toString` to see how its type is narrowed.

This `toString` function looks very similar to what you've seen in this chapter. It switches over the discriminator of `Tree` type and returns a different string for each case. Note that the type of `tree` is narrowed in each `case` of the `switch` statement.

```
abstract class Tree {
    abstract toString(): string;
}

class Empty extends Tree {
    toString() {
        return `Empty`;
    }
}

class Leaf extends Tree {
  constructor(private value: number) {
    super();
  }

  toString() {
      return `Leaf(${this.value})`;
  }
}

class Node extends Tree {
  constructor(
    private left: Tree,
    private right: Tree,
  ) {
    super();
  }

  toString() {
      return `Node(${this.left.toString()}, ${this.right.toString()}})`;
  }
}
```



UML diagram representation of the above class hierarchy.

As you can see, the OOP approach is very different. Instead of defining a standalone function, we've added an abstract method to the `Tree` abstract class. By doing that, we declare that all implementations of `Tree` have to

implement this method. What's more, when calling `toString` on a `Tree` object, the correct implementation will be chosen at runtime based on its concrete type. There is no `switch` statement. Instead, we're taking advantage of polymorphism in JavaScript.

Each concrete class provides its own implementation of `toString`. The implementations are very similar to those used in specific `case` statements in the previous approach, except they refer to `this` instead of the function argument.

Comparison

As you can see, both approaches can be used to achieve the same result. However, there are a few major differences. First, the OOP approach is much more verbose. As you can see, much more code had to be written to achieve the same result.

The most important difference is how these two approaches cope with change. Imagine that we want to support a new operation on the `Tree` (e.g., `sum` that returns the sum of all nodes in the tree). In OOP approach, we need to modify every concrete class extending from `Tree`. This can be difficult, especially if we don't own all of these classes. In the ADT approach, all we need to do is write another standalone function with a `switch` statement.

On the other hand, imagine that we want to add a new type of tree node (e.g., `NodeWithValue`). As a consequence, we need to modify all `Tree` operations. In the OOP approach, all we need to do is create a new subclass (`NodeWithValue`) and implement all abstract methods. However, in the ADT approach, we have to add a new `case` to every `switch` statement that switches over the discriminator property. It might be tricky, especially if we don't own all of the operations.

As you can see, both approaches react to change differently. I personally prefer the ADT approach, since it's terser and more in line with the functional programming paradigm.

Introduction

This chapter walks through several advanced types and concepts in TypeScript.

We'll start with *conditional types*. After a detailed introduction, we'll look at built-in conditional types and their implementations. This will provide us with great learning material to properly understand the concept.

Next, we'll talk about *mapped types*. Again, we'll go through the built-in types and learn some of their useful applications.

Next, we'll cover *type guards*, mechanisms for type-safe type narrowing. We'll learn how to build our own guards.

The chapter ends with a lesson about *nominal types*, a very useful tool that isn't part of TypeScript but can be emulated easily.

Conditional Types Introduction

This lesson introduces conditional types.

WE'LL COVER THE FOLLOWING

- Introduction
- Distributive conditional types
- Exercise

Introduction

Conditional types are *type constructors* that create a type based on some condition. In other words, they are a special kind of generic type where you can inspect the type argument and "return" a different type based on whether or not it satisfies a given type condition.

There is only one kind of *condition* allowed in conditional types and you can check whether the provided type argument is assignable to some type. You can think of type assignability the same way you'd think of value equality in a regular condition. In fact, even the syntax for conditional types is similar to regular ternary expressions.

```
type IsString<T> = T extends string ? true : false;

type A = IsString<string>; // A === true
type B = IsString<"abc">; // B === true
type C = IsString<123>; // C === false
```

In the above example, `IsString` is a conditional type. It examines the type argument `T` by checking if it is assignable to the `string` type. If the condition is true, the `true` Boolean literal type is returned (not a `true` Boolean value). Otherwise, `false` Boolean literal type is returned. Notice how type `B` is equal to `true`. This is because the `"abc"` string literal is also assignable to type

`string`.

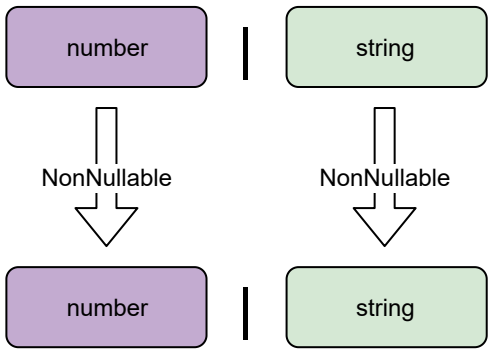
Distributive conditional types

One very important aspect of conditional types is that they're distributive. The easiest way to understand this is by looking at an example.

```
type NonNullable<T> = T extends null | undefined ? never : T;
```

`NonNullable` is an example of a built-in conditional type that comes with TypeScript 2.8. It's a very useful type that takes any type and returns a non-optional or a type with `null` and `undefined` removed from the domain of that type). You can use this type to annotate a generic function that requires a non-nullable type as a parameter (e.g., `function foo<T>(bar: NonNullable<T>) {}`).

To understand how this works, you need to know that conditional types are distributive over union type members. This means that if you apply `NonNullable` to a union type, the result will be a union of the results of `NonNullable` applied to all union members.

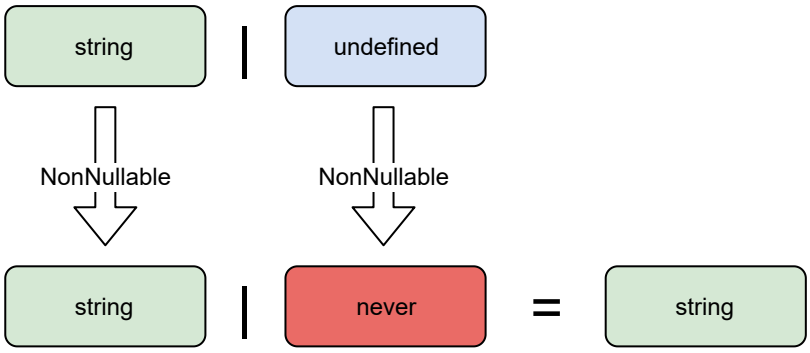


NonNullable distributed over number | string

```
type Foo = NonNullable<number | string>; // Foo is number | string
```

Of course, `NonNullable<number | string>` is still `number | string`. Let's look at a more meaningful example.

```
type Foo = NonNullable<string | undefined>; // Foo is string
```



NonNullable distributed over string | undefined

1. `NonNullable<string | undefined>` distributes over union members, so we need to evaluate `NonNullable<string>` and `NonNullable<undefine>`.
2. `NonNullable<string>` is `string` because `string` doesn't extend `null | undefined`.
3. `NonNullable<undefined>` is `never` because `undefined` extends `null | undefined`.
4. The result is `string | never` which is the same as a `string`.

But wait, what is the `never` type? It's a special type that has no instances. In other words, it's not possible to create an instance of this type. If we look at it from the perspective of set theory, `never` is associated with an empty set. Therefore, `string | never` is just a `string` because a union of any set with an empty set is just that set.

We'll look at more built-in conditional types in the next lesson.

Exercise

Define two types, `Not` and `Or`.

`Not`, given a `true` Boolean literal type, should return `false`.

`Or` should act like the logical OR operator, but on Boolean literal types.

Note that the system can only verify that your code compiles without errors. To check your solution, click on ***Show solution*** *and compare the code.*

```
// Define Not and Or
type Not<T> = never;
type Or<T> = never;
```

```
// The following assignments should compile correctly
const foo: Not<true> = false;

const foo: Not<false> = true;

const bar: Or<[true, false]> = true;
const bar: Or<[false, false]> = false;
```

The next lesson walks you through several examples of built-in conditional types.

Built-in Conditional Types

This lesson walks through several of TypeScript's built-in conditional types and explains them in detail.

WE'LL COVER THE FOLLOWING

- `Exclude`
- `Extract`
- `Omit`

`Exclude`

`Exclude` takes a union type, `T`, and removes the members that are assignable to `U`. It's another example of the distributiveness of conditional types. For all union members that extend `U`, the conditional expression will return `never`. The result will be a union of the remaining members.

`Exclude` is particularly useful when combined with the `keyof` operator. If you're not familiar with it, `keyof` takes an interface and returns a union type of literal types representing names of properties of this interface.

As you can see in *Example 3*, `Exclude` can be used to allow passing all object properties to a function *except* some specific property (or properties). In this example, `safeSetProp` is a generic function with two type parameters, `T` and `K`. We require that `K` extends `Exclude<keyof T, 'id'>` as it must be a literal type representing one of the properties except for `'id'`.

```
type MyExclude<T, U> = T extends U ? never : T;

// Example 1
type AB = Exclude<'A' | 'B' | 'C', 'C'>; // 'A' | 'B'

// Example 2
type SomeNumbers = MyExclude<'A' | 'B' | 1 | 2, string>; // 1 | 2

// Example 3
interface Person {
```

```
    id: number;
    name: string;
    age: number;

}

function safeSetProp<T, K extends MyExclude<keyof T, 'id'>>(obj: T, key: K, value: T[K]) {
    obj[key] = value;
}

declare const obj: Person;
safeSetProp(obj, 'name', 'Miłosz');
safeSetProp(obj, 'id', 100); // ● Error!
```

Run the code to see the error.

`Extract`

`Extract` is the exact opposite of `Except`. It will only pick those union members of `T` that **are** assignable to `U`.

Example 3 shows how `Extract` can be used to limit property keys passed to a function to only string keys. In the example `keyof StrangeObj` would return a union of `'foo' | 'bar' | 1 | 42`. `Extract<keyof StrangeObj, string>` eliminates non-string literal types and returns `'foo' | 'bar'`.

```
type MyExtract<T, U> = T extends U ? T : never;

// Example 1
type AB = MyExtract<'A' | 'B' | 'C', 'C'>; // 'C'

// Example 2
type SomeNumbers = MyExtract<'A' | 'B' | 1 | 2, string>; // 'A' | 'B'

// Example 3
interface StrangeObj {
    foo: string;
    bar: number;
    1: string;
    42: number;
}

function setStringProp<T, K extends MyExtract<keyof T, string>>(obj: T, key: K, value: T[K])
    obj[key] = value;
}

declare const obj: StrangeObj
setStringProp(obj, 'bar', 1);
setStringProp(obj, 42, 1); // ● Error!
```

Run the code to see the error.

`Omit`

`Omit` is a useful type that lets you take an object type and remove some properties from it.

It's defined using `Pick` which we can't discuss yet because it is a mapped type. For now, just assume that `Pick<T, U>` selects some properties from type `T` based on union members of `U`. By feeding `Exclude<keyof T, K>` as `U`, we will end up with all properties from `T` except those in `K`.

Let's break down the below example:

1. `Omit<Person, 'id'>` will resolve to `Pick<Person, Exclude<keyof Person, 'id'>>`.
2. `keyof Person` returns `'id' | 'name' | 'age'`.
3. `Exclude<keyof Person, 'id'>` returns `'name' | 'age'`.
4. `Pick<Person, 'name' | 'age'>` returns an object type `{ name: string; age: number; }`.

```
interface Person {
    id: number;
    name: string;
    age: number;
}

type AnonymousPerson = Omit<Person, 'id'>;

const anonymousPerson: AnonymousPerson = { id: 1, name: 'John', age: 33 }; // Error!
```

Hover over `AnonymousPerson` to see the inferred type. Run the code to see the error when trying to assign an object with omitted property.

The next lesson talks about the `infer` keyword which makes conditional types even more powerful.

The `infer` Keyword

The lesson goes through some of the remaining built-in conditional types and introduces the `infer` keyword.

WE'LL COVER THE FOLLOWING

- `Parameters`
- `Return`

`Parameters`

The `Parameters` type takes a function type and returns a tuple type representing types of all parameters of the function. Sounds magical, doesn't it?

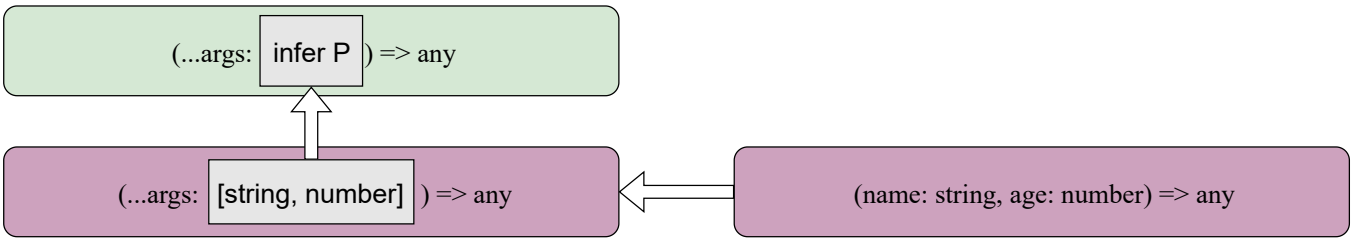
```
type Parameters<T extends (...args: any) => any> = T extends (...args: infer P) => any ? P :

const sayHello = (name: string, age: number) => `Hello ${name}, your age is ${age}`;
type SayHelloParams = Parameters<typeof sayHello>; // [string, number]
```

Hover over `SayHelloParams` to see the inferred type.

Let's break down this definition. First, the `Parameters` type has a type argument constraint that says that `T` must be a function type (`(...args: any) => any`). Next, inside the conditional expression we say that `T extends (...args: infer P) => any`. `P` is not a type argument, it's like a **type variable** which will hold the actual type of function parameters (as a tuple). The `infer` keyword lets us *unwrap* a type argument and pick a consistent part.

Another way to think about this is that `T` is matched against the `(...args: infer P) => any` *pattern*. If it matches the pattern, then `P` becomes the type that is at the same place in `T` as `infer P` in the pattern. For example, let's assume that `T` is `(name: string, age: number) => string`. It gets matched against `(...args: infer P) => any`. Since it can also be represented as `(...args: [string, number]) => string`, it matches the pattern. The

equivalent of `infer P` in `T` is `[string, number]`, so `P` gets resolved to `[string, number]` inside the positive branch of the conditional expression.



Symbolic illustration of how the type (in purple) is matched against the "pattern" (in green).

Having `P` contain the type we're interested in, we simply return it in the positive branch of the condition. We return `never` in the negative branch; we'll never enter this branch, as guaranteed by the constraint on `T`.

`Return`

`ReturnType` works in a similar way. It accepts a function type, `T`, and it extracts the return type of the function using the `infer` keyword. This time `infer R` is situated in another place of the pattern. When `T` is matched against the pattern, `R` gets resolved to the return type of the function.

```
type ReturnType<T extends (...args: any) => any> = T extends (...args: any) => infer R ? R :

const sayHello = (name: string, age: number) => `Hello ${name}, your age is ${age}`;
type SayHelloReturnType = ReturnType<typeof sayHello>; // string
```

Hover over `SayHelloReturnType` to see the inferred type.

One example of where `ReturnType` is useful is in Redux where you define action creators and reducers. A reducer accepts a state object and an action object. You can use `ReturnType` of the action creator to type the action object.

```
function fetchDataSuccess(data: string[]) {
  return {
    type: 'fetchDataSuccess',
    payload: data
  }
}

function reduceFetchDataSuccess(
   state: State,
   { payload }: ReturnType<typeof fetchDataSuccess>
```

```
) {}
```

Hover over `payload` to see the inferred type.

The next lesson walks through an example of using conditional types in practice.

Conditional Types Example - React Component Props

This lesson shows a real-life usage of conditional types in React.

WE'LL COVER THE FOLLOWING

- Detecting a React component
- Extracting component props type with `infer`

Detecting a React component

How can we take advantage of conditional types and use them to extract the types of React component properties? Let's create a conditional type that checks whether a given type is a React component.

```
type IsReactComponent<T> =
  T extends React.ComponentType<any> ? "yes" : "no";
```

`IsReactComponent` takes a type argument `T` and checks whether it extends `React.ComponentType`. If yes, it returns a `"yes"` string literal type. Otherwise, it returns a `"no"` string literal type.

Since `React.ComponentType` is a generic type, we had to specify the type parameter for `React.Component`, so we provided `any`. However, TypeScript lets us do something even cooler; we can use the `infer` keyword instead.

```
type IsReactComponent<T> =
  T extends React.ComponentType<infer P> ? "yes" : "no";
```

Extracting component props type with `infer`

`infer` creates a new type variable, `P`, that will store the type parameter of `T` if it indeed extends `React.Component`. In our case, it will be exactly what we're looking for, the type of `props`! So, instead of returning a `"yes"` literal type, it

simply returns `P`.

```
type IsReactComponent<T> =
  T extends React.ComponentType<infer P> ? P : "no";
```

Now, we can assume that this type will only be used with actual React components. We'd like the compilation to fail otherwise. Instead of returning `"no"`, we want it to return the `never` type. It's a special type that is intended exactly for these situations. We return `never` when we don't want something to happen. If a variable has a `never` type, then nothing can be assigned to it.

```
type PropsType<C> =
  C extends React.ComponentType<infer P> ? P : never;

class Article extends React.Component<{ content: string }> {
  render = () => this.props.content; // This would normally be some JSX.
}

type ArticleProps = PropsType<typeof Article>; // { content: string; }
```

Hover over `ArticleProps` to see the inferred type.

And that's it! `PropsType` takes a type argument, `C`, and matches it against `React.ComponentType<infer P>`. If there is a match, the type argument of `React.ComponentType` is stored inside `P` and `P` is returned. Otherwise, `never` is returned.

Let's break down the above example:

1. `Article` extends `React.Component<{ content: string }>`, which extends `React.ComponentType<{ content: string }>`.
2. `React.ComponentType<{ content: string }>` matches `React.ComponentType<infer P>`.
3. `{ content: string }` is stored inside `P` and returned.
4. `ArticleProps` is inferred to `{ content: string }`.

Note that we have to pass the `typeof Article` instead of `Article` to `PropsType`. `Article` type represents the shape of instances of the `Article` class. On the other hand, `typeof Article` represents the constructor function of the `Article` class. We have to use the latter because of how `React.ComponentType` is defined.

The next lesson introduces another advanced type concept, mapped types.

Mapped Types Introduction

This lesson introduces the concept of mapped types.

WE'LL COVER THE FOLLOWING

- Overview
- Mapped type example - `Readonly` type
- Deep immutability

Overview

Mapped types are another way to transform types. In simple words, they let you take an object type (an interface) and return a new type with each property transformed in some way.

Mapped type example - `Readonly` type

The best example for understanding mapped types is the built-in `Readonly` type. As you can see, `Readonly` is a generic type. It takes an object type `T`, enumerates its properties (`P in keyof T`), and adds the `readonly` modifier to each property. You can think of mapped types as an equivalent of the `for..of` loop in the world of types.

```
interface Person {
    name: string;
    age: number;
}

type Readonly<T> = {
    readonly [P in keyof T]: T[P];
};

type ReadonlyPerson = Readonly<Person>;
```

Hover over `ReadonlyPerson` to see the inferred type.

The syntax reminds the one for defining a regular interface. However, instead of specifying several properties, you define all of them at once by using the `[P in keyof T]` syntax (*index type query operator*). `P` becomes the literal type representing each property, and the value of each property is simply `T[P]` (*indexed access operator*).

Deep immutability

`Readonly` type is very helpful when dealing with immutable data structures (e.g., when working with Redux). Bear in mind that it only provides shallow immutability, meaning nested properties will not become `readonly`. You can achieve better results with the following `DeepReadonly` type.

```
type DeepReadonly<T> = {
  readonly [P in keyof T]: DeepReadonly<T[P]>;
};

type Employment = DeepReadonly<{
    person: Person;
    company: string;
}>;

interface Person { name: string; }

declare const employment: Employment;
employment.person.name = 'Milosz'; // ● Error!
```

Run the code to see the error.

As you can see, mapped types can be recursive. For each property of `T` we map it to a read-only property and we declare its type as `DeepReadonly`. If the original property type is an object type, its properties will also be marked with `readonly`. Otherwise, the type will stay unmodified.

Unfortunately, this type does not provide immutability for nested arrays, Maps, Sets, or some other types. It's actually quite complex to properly implement such types, so it's best to use a library instead (like this one).

The next lesson walks you through several examples of built-in mapped types.

Built-in Mapped Types

This lesson walks through some of the built-in mapped types and explains them in detail.

WE'LL COVER THE FOLLOWING

- `Partial`
- `Required`
- `Pick`
- `Record`
- Exercise

`Partial`

`Partial<T>` returns a type that has the same properties as `T` but all of them are optional. This is most useful when the `strictNullChecks` flag is enabled. `Partial` works on a single level, meaning it doesn't affect nested objects. A good use case for `Partial` is when you need to type a function that lets you override default values of properties of some object.

As you can see, `Partial` is defined in a similar way to `Readonly`. Instead of the `readonly` modifier, the optionality modifier `?` is appended to each property of `T`.

```
interface Settings {
  width: number;
  autoHeight: boolean;
}

type Partial<T> = {
    [P in keyof T]?: T[P];
};

const defaultSettings: Settings = {};

function getSettings(custom: Partial<Settings>): Partial<Settings> {
  return { ...defaultSettings, ...custom };
}
```

Try code completion on `custom` to see the types of the properties.

Required

`Required<T>` removes optionality from `T`'s properties. It demonstrates how mapped types can not only add modifiers to properties but also remove them with the `-` syntax.

```
type Required<T> = {
    [P in keyof T]-?: T[P];
};

type Foo = Required<{ name?: string }>; // { name: string; }
```

Hover over `Foo` to see the inferred type.

Pick

`Pick` lets you create an interface that contains only a subset of properties of `T`.

Contrary to previous examples of mapped types, `Pick` doesn't use `[P in keyof T]` to iterate over properties. Instead, `[P in K]` is used, where `K` is a type argument to `Pick`. In fact, any union type can be placed on the right side of `in`. Here, `K` is constrained to extend `keyof T` to make sure that only existing properties of `T` are present in the result type.

```
interface Person {
  name: string;
  age: number;
  id: number;
}

type Pick<T, K extends keyof T> = {
    [P in K]: T[P];
};

type NameAndAge = Pick<Person, 'name' | 'age'>; // { name: string; age: number; }
```

Hover over `NameAndAge` to see the inferred type.

Record

Finally, `Record` is an interesting example of a mapped type that doesn't actually map over any source type. Instead, it creates a completely new object

type out of *thin air*.

`Record` takes two type arguments: `K` representing property names in the resulting type, and `T` representing the type of values of these properties.

```
type Record<K extends keyof any, T> = {
    [P in K]: T;
};

type StringDictionary = Record<string, string>; // { [x: string]: string; }
type ABCNumbers = Record<'a' | 'b' | 'c', number>; // { a: number; b: number; c: number; }
```

Hover over `ABCNumbers` to see the inferred type.

Exercise

Define a type called `ReturnTypes` that takes an interface with methods and returns an interface with properties typed using return types of these methods.

Note that the system can only verify that your code compiles without errors. To check your solution, click on ***Show solution*** *and compare the code.*

```
type Test = ReturnTypes<{
    a: string;
    b: (a: number) => number;
}>; // { a: never; b: number; }
```

The next lesson introduces another advanced type mechanism, type guards.

Type Guards

This lesson introduces the concept of type guards.

WE'LL COVER THE FOLLOWING

- Overview
- Nullable type guard
- `typeof` type guard
- `instanceof` guard

Overview

Type guard is not really a *type* so much as it is a mechanism that *narrows types*.

TypeScript soars at analyzing your code and deducing information based on it. Oftentimes, TypeScript can analyze a conditional expression (or an `if` statement) and narrow the type of the tested value inside the branches of the condition. We've already seen this happen with discriminated unions. This mechanism is referred to as *type guard*.

Nullable type guard

We've already seen the nullable type guard in action when discussing `strictNullChecks`. Nullable type guard removes optionality from the type of some value if this value is previously tested for nullability. In the below example, if the `name` is truthy (i.e., not `null` or `undefined`), its type is narrowed to `string` inside the positive branch of the `if` statement.

```
function sayHello(name?: string) {
  if (name) {
    console.log(`Hello, ${name.toUpperCase()}!`);
  } else {
    console.log("Hello!");
```

```
  }
}
```

Hover over `name` inside the first `if` branch to see how its type is narrowed.

`typeof` type guard

Sometimes it's possible to check the type of a value at runtime. This type guard narrows the type of checked variable to the type it's checked against. This trick only works with JavaScript primitive types (string, number, undefined, null, Boolean, and symbol). You cannot use it to check the type of an interface because this information is erased at runtime.

In the below example, `width` can be either a `number` or a `string`. Not only the type of `width` is narrowed to a `number` in the first branch, but it's also narrowed to a `string` in the second branch.

```
class Properties {
  width: number = 0;

  setWidth(width: number | string) {
    if (typeof width === "number") {
      this.width = width;
    } else {
      this.width = parseInt(width);
    }
  }
}
```

Hover over `width` within both branches of the `if` statement to see the inferred types.

`instanceof` guard

This type guard can be used with classes. Contrary to interfaces, the type information for classes is retained at runtime. In other words, you can check if some object is an instance of some class at runtime. This type guard will narrow the type of that object to this class inside the conditional expression.

```
class Person {
  constructor(public name: string) {}
}

function greet(obj: any) {
  if (obj instanceof Person) {
    console.log(obj.name);
  }
}
```

Hover over `obj` within the `if` statement to see the inferred type.

The next lesson discusses how to create your own type guards.

Custom Type Guards

This lesson explains custom type guards and how they're useful.

WE'LL COVER THE FOLLOWING

- Overview
- Runtime checks

Overview

TypeScript lets you define your own type guards. A custom type guard is a Boolean-returning function that can additionally assert something about the type of its parameter. If the function is called in the context of a conditional expression, the type of the passed value is narrowed to the asserted type inside the positive branch of the conditional.

Below, you can see an example of an `isArticle` type guard. It's a function that takes an object and checks if it's an instance of `Article` by checking for the presence of certain properties. Later, when the function is used inside the `if` statement, the type of `body` is narrowed to `Article` in the first branch of the conditional statement.

```
interface Article {
  title: string;
  content: string;
}

function isArticle(object: any): object is Article {
  return "title" in object && "content" in object;
}

fetch("http://example.com")
  .then(response => response.json())
  .then(body => {
    if (isArticle(body)) {
      return body.title;
    } else {
      throw new Error("This is not an article");
```

```
  }
});
```

Hover over `body` on line 14 to see how it's type is narrowed.

Runtime checks

Custom type guards are often used when dealing with data coming from the outside world. TypeScript doesn't have any runtime checks built-in. For example, if we fetch some data from the backend and assert its type as `Article` then TypeScript has no way to actually verify it, it has to believe us. A custom type guard is a safer alternative to type assertion (a cast) as it allows us to perform a runtime check before assertion. However, the correctness of the runtime check is our responsibility; if the check is not exhaustive then we may end up with a runtime error anyway.

For example, in the above piece, we don't check that the type of the `title` property is `string`. The backend might return an object that has the `title` property with the value `42`. Later, we might try to call the `toLowerCase()` method on the `body.title` because we think it's a string. At this point, we would get a runtime error.

In summary, type guards are like customizable type assertions where you can perform a runtime check to ensure the validity of the assertion. It's up to you to write the check correctly and to ensure that it's sufficient.

The final lesson introduces one more advanced type concept, nominal types.

Nominal Types

This lesson talks about nominal types, a concept that is not part of TypeScript, but can be emulated easily.

WE'LL COVER THE FOLLOWING

- Structural vs nominal typing
- Branded types
- Generalizing nominal types
- Example: non-empty strings

Structural vs nominal typing

The majority of programming languages traditionally associated with static typing (Java, C#, C++) follow an approach called *nominal typing*. In a nominal type system every type is unique. Imagine that you have created two interfaces with exactly the same properties. In TypeScript, it's perfectly fine to assign an instance of one of these interfaces to a variable typed using the other one. However, in languages with nominal typing, that wouldn't be possible. TypeScript doesn't use nominal typing, instead is uses *structural typing*.

```
interface Cat {
    name: string;
    breed: string;
}

interface Dog {
    name: string;
    breed: string;
}

const cat: Cat = { name: 'Filemon', breed: 'Chartreux' };
let dog: Dog = cat; // ✅ No error
```

Run the code to verify that no error is produced.

The TypeScript is different because of the need for compatibility with JavaScript. With nominal typing, even the line below would result in an error:

```
const cat: Cat = { name: 'Filemon', breed: 'Chartreux' };
```

This is because the right-hand side of the assignment is an object of object literal type `{ name: string; breed: string; }`, a type that is nominally different from type `Cat`. Nominally typed language would therefore not allow such an assignment.

Branded types

However, nominal typing is sometimes useful. Imagine writing an application that loads many entities from a database. Each entity has some identifier (`id`) represented as a `string`. It might happen that you accidentally pass a `userId` where an `articleId` is needed and there is nothing in the type system that would prevent this.

With nominal typing, you could create `UserId` and `ArticleId` types that are effectively aliases for `string`, but since every type is unique in nominal typing, the type checker would let you pass an instance of `ArticleId` where `UserId` is required.

In TypeScript, it is possible to achieve suthish behavior with *branded types*.

```
type UserId = string & { __brand: "UserId" };
type ArticleId = string & { __brand: "ArticleId" };

declare const userId: UserId;

function getArticle(articleId: ArticleId) {}

getArticle(userId); // ● Error!
```

Run the code to see the error.

You can create a branded type by intersecting it with an object literal with one string literal property, `__brand`. The name of the property is just a convention,

any name should work. `UserId` is not assignable to `ArticleId` because the types of their `__brand` properties don't match. Therefore, it's very important to make sure that string literals used as `__brand` are unique across the whole application.

Generalizing nominal types

It's possible to generalize this mechanism. Below you can see the `Nominal` generic type which takes any type `T` and makes it a branded type by intersecting it with an object literal with `__brand` typed as `K`.

```
type Nominal<T, K extends string> = T & { __brand: K };

type UserId = Nominal<string, "UserId">;
type ArticleId = Nominal<string, "ArticleId">;
```

Example: non-empty strings

An example of using nominal types is making the type system help you detect a situation when someone is passing an empty string where a non-empty string is expected.

```
type Nominal<T, K extends string> = T & { __brand: K };

type NonEmptyString = Nominal<string, "nonempty">;

const isNonEmpty = (s: string): s is NonEmptyString => s.length > 0;

const foo = (s: NonEmptyString) => console.log(s);

const bar = "abcd";
foo(bar); // error!

if (isNonEmpty(bar)) {
  foo(bar); // OK
}
```

Run the code to see the error.

`NonEmptyString` is a branded `string`. The only way to create an instance of `NonEmptyString` (aside from type assertion) is by using a custom type guard, `isNonEmpty`. The type guard checks whether the string is indeed empty, in which case, it marks the string as `NonEmptyString`. `foo` is a function that

which case, it marks the string as `NonEmptyString`. `foo` is a function that requires a `NonEmptyString`. The only way to call this function is to wrap it in the `isNonEmpty` type guard, which effectively forces the caller to only call the function if the string is indeed non-empty.

Introduction

This last chapter focuses on practical applications of TypeScript.

The first three lessons deal with the topic of debugging TypeScript compile errors. Error messages in TypeScript are infamous for their complexity. The lessons in this chapter aim to teach you how to approach them by providing some examples.

The second part of the chapter contains examples showing how TypeScript is used in some libraries. The advanced typing scenarios in this chapter are a perfect practice ground for the concepts taught throughout the course.

Debugging Errors: Type Assignability

This lesson advises you on how to approach one of the most common TypeScript compile errors.

WE'LL COVER THE FOLLOWING

- Overview
- Deeply nested properties
- Simplified error messages
- Exercise

Overview

The most common TypeScript error is related to type assignability. We get them whenever we want to use a value of some type when a different type is required. These are the bread and butter of TypeScript. The main goal of the type checker is to ensure that you provide values with correct types wherever required.

Most of the time, type assignability errors are straightforward. The error message says that the type of the right-hand side of the assignment is not assignable to the type of the left-hand side.

```
// ● Type '5' is not assignable to type 'string'.
const foo: string = 5;
```

Run the code to see the error.

Deeply nested properties

However, these errors can get tricky, especially with type incompatibility of deeply nested properties.

```
export interface Foo {
    a: {
        b: {
            c: {
                d: number;
            }
        }
    }
}

function woooot(foo: Foo) {}

const foo = {
    a: {
        b: {
            c: {
                d: 'abc',
            }
        }
    }
};

 // ● Error!
woooot(foo);
```

Run the code to see the error.

Let's look at the error produced by TypeScript for the above piece of code.

```
Argument of type '{ a: { b: { c: { d: string; }; }; }; }' is not assignabl
e to parameter of type 'Foo'.
  Types of property 'a' are incompatible.
    Type '{ b: { c: { d: string; }; }; }' is not assignable to type
'{ b: { c: { d: number; }; }; }'.
      Types of property 'b' are incompatible.
        Type '{ c: { d: string; }; }' is not assignable to type
'{ c: { d: number; }; }'.
          Types of property 'c' are incompatible.
            Type '{ d: string; }' is not assignable to type '{ d: numbe
r; }'.
              Types of property 'd' are incompatible.
                Type 'string' is not assignable to type 'number'.
```

As you can see, the error messages can get quite complex even for such a simple mistake. Let's try to break it down.

```
Argument of type '{ a: { b: { c: { d: string; }; }; }; }' is not assignabl
e to parameter of type 'Foo'
```

The top sentence tells us that the inferred side of the provided function argument `foo` is not assignable to the expected type `Foo`. This is good to know, but why are they not assignable?

```
Types of property 'a' are incompatible.
```

The next sentence explains the reason: types of a property called `a` in both types are not compatible. Good, let's see why.

```
Type '{ b: { c: { d: string; }; }; }' is not assignable to type
'{ b: { c: { d: number; }; }; }'.
```

So, the inferred type of property `a` in `foo` is `{ b: { c: { d: string; }; }; }` while the type of `a` in `Foo` is `{ b: { c: { d: number; }; }; }`. I bet you can already spot the mistake, but imagine these types having more properties. It could be tough to spot the difference with your own eyes. Fortunately, TypeScript helpfully says that `Types of property 'b' are incompatible.`.

The subsequent error messages go deeper and deeper and finally we learn that `Types of property 'd' are incompatible` and `Type 'string' is not assignable to type 'number'`.

Finally, we've found the problem! If you think about it, the last two lines were critical to figuring out what happened. However, sometimes the type might be so complex that you have no idea where to look for the incompatible property. Therefore, sometimes it might be necessary to go through the whole trace of incompatible types.

Simplified error messages

Some good news: starting from TypeScript 3.7, the checker can sometimes simplify and flatten the error message. In particular, the above message would show as below.

```
Argument of type '{ a: { b: { c: { d: string; }; }; }; }' is not assignabl
```

```
e to parameter of type 'Foo'.


  The types of 'a.b.c.d' are incompatible between these types.
    Type 'string' is not assignable to type 'number'.(2345)
```

However, you'll only get the simplified messages in some cases, so it's still important to be able to read deeply nested error messages.

Exercise

Fix the compile error in the below code piece.

```
interface MyEvent {
    target: {
        id?: string;
    }
}

interface Props {
    width: number;
    height: number;
    callbacks: {
        onSuccess: (event: MyEvent) => void;
    }
}

const component: (props: Props) => string =
    (props) => `<div width="${props.width}"></div>`;

const props = {
    width: 100,
    height: 200,
    callbacks: {
        onSuccess: (event: { target: { id: string } }) => { console.log(event.target.id); }
    }
};

component(props);
```

The next lessons goes through an example of fixing another class of errors.

Debugging Errors: Narrowing Types

This lesson walks through the debugging of a compile error that can be addressed by appropriate type narrowing.

WE'LL COVER THE FOLLOWING

- Overview
- Digging deeper
- Type guard to the rescue

Overview

Another error that we'll look at is an example of a situation when TypeScript is not able to narrow the type of a variable, even though it might be obvious from the perspective of the programmer.

```
interface Person { name: string; employer?: string; };

function getEmployers(persons: Person[]): string[] {
    return persons
        .filter(person => person.employer !== undefined)
        .map(person => person.employer); // ● Error!
}
```

Run the code to see the error.

The above code will result in the following error message.

```
Type '(string | undefined)[]' is not assignable to type 'string[]'.
  Type 'string | undefined' is not assignable to type 'string'.
    Type 'undefined' is not assignable to type 'string'.ts(2322)
```

Digging deeper

Let's break down the error message.

```
Type '(string | undefined)[]' is not assignable to type 'string[]'.
```

The error message points to the line containing the `return` statement. It says that that the inferred type of the expression passed to the `return` statement `(string | undefined)[]` does not match the expected return type `string[]`. The subsequent lines aim to provide some explanation on why these two types don't match. Both types are array types, so TypeScript compares the type of the array element in both. `string | undefined` is indeed not assignable to `string` because `undefined` is not assignable to `string`. To summarize, we're trying to return an array of possibly undefined string values, where a value of always defined string values is expected.

"But we filter out undefined values," one might say! However, TypeScript has no way of knowing how the predicate provided to `filter` method affects the type of the array. Even though it's obvious that `person.employer` will be defined inside `map`, its type is still `string | undefined`. We can't expect TypeScript to understand the semantics of `Array.filter`, so we need to help it a bit.

Type guard to the rescue

Fortunately, there is a way to fix this in an elegant way. Interestingly, there are two overloads for `Array.filter`:

```
filter(callbackfn: (value: T, index: number, array: T[]) => unknown, thisA
rg?: any): T[];
filter<S extends T>(callbackfn: (value: T, index: number, array: T[]) => v
alue is S, thisArg?: any): S[];
```

The first one is the more obvious one. You've probably been using it all the time without even knowing about it. The second one is more interesting. It's a generic function with type argument `S`, where `S` is a subtype of `T`. The type of the `callbackFn` parameter is `(value: T, index: number, array: T[]) => value is S`; it's a type guard! All array elements for which `callbackFn` returns `true` can be typed as `S` instead of `T`. Therefore, the return type of the whole method is `S[]`.

All we need to do is provide a type guard version of the predicate, where we

All we need to do is provide a type guard version of the predicate, where we tell TypeScript that if `person.employer !== undefined` then the type of `person` can be narrowed to `Required<Person>`.

```
interface Person { name: string; employer?: string; };

function getEmployers(persons: Person[]): string[] {
    return persons
        .filter((person): person is Required<Person> => person.employer !== undefined)
        .map(person => person.employer);
}
```

Run the code to verify that it compiles successfully.

This example shows how it's sometimes required to provide the type checker a little help to successfully resolve an error.

The next lesson discusses coping with unintuitive error messages.

Debugging Errors: Unintuitive Error Message

This lesson walks you through debugging a complex and unintuitive error message.

WE'LL COVER THE FOLLOWING

- Overview
- Digging deeper
- Fixing the error

Overview

In this example, we'll look at a complex and unintuitive error message.

Below you can find a short piece of code using the RxJS library. In short, RxJS is a library for creating and manipulating streams of data. The piece of code below is supposed to create a stream of five numbers, map it into a stream of those numbers multiplied by two, and finally return a stream that emits only the last number.

```
const source = of(1, 2, 3, 4, 5).pipe(
  map(x => x * 2),
  last // Error!
);
```

When compiled, this code will yield a large and complex error message.

```
Argument of type '{ <T, D = T>(predicate?: null | undefined, defaultValu
e?: D | undefined): OperatorFunction<T, T | D>; <T, S extends T>(predicat
e: (value: T, index: number, source: Observable<T>) => value is S, default
Value?: S | undefined): OperatorFunction<...>; <T, D = T>(predicate: (valu
e: T, index: number, source: Observable<.....' is not assignable to parame
ter of type 'OperatorFunction<unknown, unknown>'.
  Types of parameters 'predicate' and 'source' are incompatible.
    Type 'Observable<unknown>' is not assignable to type 'null | undefine
d'
```

```
      Type 'Observable<unknown>' is not assignable to type 'null'.ts(2345)
```

Digging deeper

Let's break down this error message.

```
Argument of type '{ <T, D = T>(predicate?: null, defaultValue?: D): Operat
orFunction<T, T | D>; <T, S extends T>(predicate: (value: T, index: numbe
r, source: Observable<T>) => value is S, defaultValue?: S): OperatorFuncti
on<T, S>; <T, D = T>(predicate: (value: T, index: number, source: Observab
le<...>) => boolean, defaultValue?: D): ...' is not assignable to paramete
r of type 'OperatorFunction<unknown, unknown>'.
```

Looks like we're trying to pass something of a very complex type `{ <T, D = T>(predicate?: null, defaultValue?: D): OperatorFunction<T, T | D>;...` where `OperatorFunction<Response, unknown>` is expected. Where does this huge type come from? When we inspect the source code of the `last` operators, we'll see three function definitions, none of them matching this lengthy type definition.

```
export declare function last<T, D = T>(predicate?: null, defaultValue?: D
): OperatorFunction<T, T | D>;
export declare function last<T, S extends T>(predicate: (value: T, index:
number, source: Observable<T>) => value is S, defaultValue?: S): OperatorF
unction<T, S>;
export declare function last<T, D = T>(predicate: (value: T, index: number
, source: Observable<T>) => boolean, defaultValue?: D): OperatorFunction<T
, T | D>;
```

Let's look closer at this type.

```
{
    <T, D = T>(predicate?: null, defaultValue?: D): OperatorFunction<T, T
| D>;
    <T, S extends T>(predicate: (value: T, index: number, source: Observab
le<T>) => value is S, defaultValue?: S): OperatorFunction<T, S>;
    <T, D = T>(predicate: (value: T, index: number, source: Observable<...
>) => boolean, defaultValue?: D): ...
}
```

Although the compiler truncated the definition, we can still see that it's a **callable interface**. A value of this type can be called in at least three ways. If you look closely, you'll notice that these three call signatures match the three

overloads of `last`! Therefore, we can see that this complex type is TypeScript's convoluted way of representing function overloads.

Fixing the error

Great, now we finally know what the error message is trying to tell us. The type we're providing, a function that returns `OperatorFunction`, doesn't match the expected type, an `OperatorFunction`. We simply forgot to call `last`.

```
const source = of(1, 2, 3, 4, 5).pipe(
  map(x => x * 2),
  last() // ✅ No errors!
);
```

With this example, I'd like to show that despite sometimes imperfect error messages coming from TypeScript, it's always possible to debug the error. Sometimes it takes a bit more time and thought to dig deeper into the error message and break it down into smaller, understandable parts.

The next lesson looks into an example of using TypeScript with the React library.

React and TypeScript: `useState` Hook

This lesson is a detailed analysis of typing the `useState` hook in TypeScript.

WE'LL COVER THE FOLLOWING

- Overview
- Reading the types
  - Overloads
  - `useState` parameters
  - `useState` return type
- Usage examples
  - Empty initial state
  - Clearable state

Overview

React hooks are a recent addition to React that make function components have almost the same capabilities as class components. Most of the time, using React hooks in TypeScript is straightforward.

However, there are some situations when a deeper understanding of a hook's type might prove useful. In this lesson, we're going to focus on the `useState` hook.

I'm going to assume that you have a basic understanding of this hook. If this is not the case, please read this first.

Reading the types

First of all, let's take a look at the type signature of `useState`. You'll see how much information you can extract solely from types, without looking at the docs or the implementation.

Overloads

```
function useState<S = undefined>(): [S | undefined, Dispatch<SetStateAction<S | undefined>>];
function useState<S>(initialState: S | (() => S)): [S, Dispatch<SetStateAction<S>>];
```

As you can see, there are two *versions* of the `useState` function. TypeScript lets you define multiple type signatures for a function as it is often the case in JavaScript that a function supports different types of parameters. Multiple type signatures for a single function are called **overloads**.

Both overloads are generic functions. The type parameter `S` represents the type of the piece of state stored by the hook. The type argument in the second overload can be inferred from `initialState`. However, in the first overload, it defaults to `undefined` unless the type argument is explicitly provided. **If you don't pass the initial state to `useState`, you should provide the type argument explicitly.**

`useState` parameters

The first overload doesn't take any parameters; it's used when you call `useState` without providing any initial state.

The second overload accepts `initialState` as a parameter. Its type is a union of `S` and `() => S`. Why would you pass a function that returns the initial state instead of passing the initial state directly? Computing the initial state can be expensive. It's only needed when the component is mounted. However, in a function component it would be calculated on every render. **Therefore, you have an option to pass a function that calculates initial state, meaning expensive computation will only be executed once, not on every render.**

`useState` return type

Let's move to the return type. It's a **tuple** in both cases. A tuple is like an array that has a specific length and contains elements with specific types.

For the second overload, the return type is `[S, Dispatch<SetStateAction<S>>]`. The first element of the tuple has type `S`, the type of the piece of state. It will contain the value retrieved from the component's state.

The second element's type is `Dispatch<SetStateAction<S>>`. `Dispatch<A>` is simply defined as `(value: A) => void`, a function that takes a value and doesn't return anything. `SetStateAction<S>` is defined as `S | ((prevState: S) => S)`. Therefore, the type of `Dispatch<SetStateAction<S>>` is actually `(value: S | ((prevState: S) => S)) => void`. It is a function that takes either an updated version of the piece of state OR a function that produces the updated version based on the previous version. In both cases, we can deduce that the second element of the tuple returned by `setState` is a function that we can call to update the component's state.

The return type of the first overload is the same, but here instead of `S`, `S | undefined` is used anywhere. If we don't provide the initial state it will store `undefined` initially. It means that `undefined` has to be included in the type of the piece of state stored by the hook.

Usage examples

Most of the time you don't need to bother with providing type arguments to `useState`, the compiler will infer the correct type for you. However, in some situations, type inference might not be enough.

Empty initial state

The first type of situation is when you don't want to provide the initial state to `useState`.

As we saw in the type definition, the type argument `S` for the parameterless defaults as `undefined`. Therefore, the type of `pill` should be inferred to `undefined`. However, due to a design limitation in TypeScript, it's actually inferred to `any`.

Similarly, `setPill`'s type is inferred to `React.Dispatch<any>`. This is really bad, as nothing would stop us from calling it with an invalid argument: `setPill({ hello: 5 })`.

```
export const PillSelector: React.FunctionComponent = () => {
    const [pill, setPill] = useState();
    return (
        <div>
            <button onClick={() => setPill('red')}>Red pill</button>
            <button onClick={() => setPill('blue')}>Blue pill</button>
```

```
            <span>You chose {pill} pill!</span>
        </div>

    );
}
```

In order to fix this issue, we need to pass a type argument to `setState`. We treat `pill` as text in JSX, so our first bet could be `string`. However, let's be more precise and limit the type to only allow values that we expect.

```
const [pill, setPill] = useState<'red' | 'blue'>();
```

Note that the inferred type of `pill` is now `"red" | "blue" | undefined` because this piece of state is initially empty. With `strictNullChecks` enabled, TypeScript wouldn't let us call anything on `pill`:

```
// 🛑 Object is possibly 'undefined'.ts(2532)
<span>You chose {pill.toUpperCase()} pill!</span>
```

...unless we check the value first:

```
// ✅ No errors!
{pill && <span>You chose {pill.toUpperCase()} pill!</span>}
```

Clearable state

Another situation where you would provide a type argument to `useState` is when the initial state is defined, but you want to be able to **clear** the state later.

```
export const PillSelector: React.FunctionComponent = () => {
    const [pill, setPill] = useState('blue');
    return (<div>
        <button onClick={() => setPill('red')}>Red pill</button>
        <button onClick={() => setPill('blue')}>Blue pill</button>
        // 🛑 Argument of type 'undefined' is not assignable
        // to parameter of type 'SetStateAction<string>'.
        <button onClick={() => setPill(undefined)}>Reset</button>
        {pill && <span>You chose {pill.toUpperCase()} pill!</span>}
    </div>);
}
```

Since the initial state is passed to `useState`, the type of `pill` gets inferred to

`string`. Therefore, when you try to pass `undefined` to it, TypeScript will error.

You can fix the problem by providing the type argument.

```
const [pill, setPill] = useState<'blue' | 'red' | undefined>('blue');
```

The next lesson talks about a more complex typing scenario in React.

React and TypeScript: `useReducer` Hook

This lesson is a detailed analysis of typing the `useReducer` hook in TypeScript. It also provides a great use case for discriminated unions and state machines.

WE'LL COVER THE FOLLOWING

- Overview
- Do we need a reducer?
- The type of `useReducer`
- Typing state
- Typing actions with discriminated unions
- Implementing the reducer
- Using the hook
- Even better state representation

Overview

In this lesson, we will implement a common data fetching scenario with the `useReducer` hook. We will see how to take advantage of TypeScript's discriminated unions to correctly type reducer's actions. Finally, we will introduce a useful pattern for representing the state of data fetching operations.

Do we need a reducer?

We will base our code on an example from the official React documentation. The demo linked from this article is a simple implementation of a very common pattern, fetching a list of items from some backend service. In this case, we're fetching a list of Hacker News article headers.

What functionality is missing in this little demo? When fetching data from the backend it's useful to indicate to the user that an operation is in progress, and

if the operation fails, to show the error to the user. Neither is included in the demo as coded.

The attached code uses `useState` hook to store the list of items after it is retrieved. We will need two additional pieces of state to implement our enhancements, a Boolean indicating whether an action is in progress and an optional string containing the error message.

We could use more `useState` hooks to store this information. There's a better way to do this, though. Notice that we're modifying multiple pieces of state at the same time as a result of certain actions. For example, when data is retrieved from the backend, we update both the data piece and the loading indicator piece. What's more, we're modifying the state in multiple places. Wouldn't it be cleaner and easier to follow if there was only a single place in the component where the state is updated?

We can achieve it by using the `useReducer` hook. It will allow us to centralize all state modification, making them easier to track and reason about.

The type of `useReducer`

Let's take a look at `useReducer`'s type signature. I simplified the code slightly by taking the initializer out of the picture.

```
function useReducer<R extends Reducer<any, any>>(
   reducer: R,
   initialState: ReducerState<R>
): [ReducerState<R>, Dispatch<ReducerAction<R>>];
```

Our hook is a function; yes, React hooks are just functions. It accepts two parameters: `reducer` and `initialState`.

The first parameter's type must extend `Reducer<any, any>`. `Reducer` is just an alias for a function type that takes a state object and an action and returns an updated version of the state object. In other words, the reducer describes how to update the state based on some action.

```
type Reducer<S, A> = (prevState: S, action: A) => S;
```

The second parameter allows us to provide the initial version of the state. `ReducerState` is a conditional type that extracts the type of the state from the

`Reducer` type.

Finally, `useReducer` returns a tuple. The first element in the tuple is the recent version of the state object. We will render our component based on the values contained in this object. The second item is a dispatch function. It is a function that will let us dispatch actions that will trigger state changes. Similar to `ReducerState`, `ReducerAction` extracts an action type from `Reducer`.

Behold the power of static typing, reading a well-typed function's signature is often enough to understand its purpose.

Typing state

Now is the time to fill in the gaps and define types representing state and actions.

It was already mentioned that apart from the data received from the server, we're also going to store a flag indicating if we're loading the data and an optional error message.

Therefore, the shape of state can be described with the following types:

```
type State = {
 data?: HNResponse;
 isLoading: boolean;
 error?: string;
}


type HNResponse = {
 hits: {
   title: string;
   objectID: string;
   url: string;
 }[]
};
```

`HNResponse` interface is based on the response received from https://hn.algolia.com/api/v1/search endpoint which we're going to use in this example. It is a free service that returns headers of Hacker News articles.

Typing actions with discriminated unions

An action is an object that represents some event in our application and results in the modification of the state. What kind of actions are there in our app?

- The first action describes the fact that the user typed some text into the search text field. This action will initiate a backend request.
- The second action will be triggered when the data from the backend is fetched. Action objects should contain this data.
- The third action will represent an error that occurred during data fetching. It should encompass the error message.

How do you represent the type of these actions in TypeScript? We can take advantage of a useful concept called discriminated union type.

```
type Action =
 | { kind: 'request' }
 | { kind: 'success', results: HNResponse }
 | { kind: 'failure', error: string };
```

`Action` is a union of three object types. What makes it special is the fact that all of those types have a common property called type. The type of this property in each interface is a different literal type. This lets us distinguish between those types.

Why is this useful? TypeScript creates automatic type guards for discriminated unions. This means that if we write an `if` statement in which we compare the type property of a given `Action` object with a specific `kind` (e.g. `success`), the type of the object inside the statement's body will be narrowed to the matching component of the union type.

For example, in the following code, accessing `action.results` will not cause a compile error because the type of action inside the body of the if statement will be appropriately narrowed!

```
function display(action: Action) {
 if (action.kind === 'success') {
   console.log(action.results);
 }
```

```
}
```

Implementing the reducer

We're all good to implement the reducer. As already mentioned, it takes a state and an action and returns an updated state.

For request actions, we're going to set the `isLoading` flag to true.

The success action will disable the `isLoading` flag and also set data to the results received from the server.

Finally, failure action will also disable the `isLoading` flag and set the error message.

```
function reducer(state: State, action: Action): State {
 switch (action.kind) {
   case 'request':
     return { isLoading: true };
   case 'success':
     return { isLoading: false, data: action.results };
   case 'failure':
     return { isLoading: false, error: action.error };
 }
}
```

Thanks to discriminated unions, we can access the action's properties inside case blocks in a type-safe way.

Using the hook

All that is left is to pass our reducer to `useReducer` hook.

```
const [{
  data,
  isLoading,
  error
}, dispatch] = useReducer(reducer, { isLoading: false });
```

I'm passing the reducer function to the hook along with the initial state which has `isLoading` set to `false` and the remaining properties `undefined`. The result is a pair with the current state object as the first element (which I'm instantly destructuring) and the dispatch function as the second element

instantly destructuring) and the dispatch function as the second element.

Next, I need to update the usage of the `useEffect` hook so that it dispatches relevant actions.

```
useEffect(() => {
    let ignore = false;
    dispatch({ kind: 'request' });
    fetch(`https://hn.algolia.com/api/v1/search?query=${query}`)
        .then(
            (response: Response) => response.json().then(
                (results: HNResponse) => {
                    if (!ignore) dispatch({ kind: 'success', results });
                }
            ),
            (error: Error) => dispatch({ kind: 'failure', error: error.mes
sage })
        );

    return () => { ignore = true; }
}, [query]);
```

Finally, we should update the JSX to take the new pieces of the state into account and show the loading indicator and the error message when available.

```
return (
    <div>
        <input value={query} onChange={e => setQuery(e.target.value)} />
        {isLoading && <span>Loading...</span>}
        {error && <span>Error: {error}</span>}
        <ul>
        {data && data.hits && data.hits.map(item => (
            <li key={item.objectID}>
            <a href={item.url}>{item.title}</a>
            </li>
        ))}
        </ul>
    </div>
);
```

Below you can find the full implementation of the component. Note that we're still using the `useState` hook to store the query. This information is completely unrelated to the rest of the state, therefore there would be no

completely unrelated to the rest of the state, therefore there would be no advantage in including it in the state managed by `useReducer`.

```
import * as React from "react";
import * as ReactDOM from "react-dom";
import { Component, useState, useEffect, useReducer } from 'react';

type HNResponse = {
  hits: {
    title: string;
    objectID: string;
    url: string;
  }[]
};

type State = {
  data?: HNResponse;
  isLoading: boolean;
  error?: string;
}

type Action =
  | { kind: 'request' }
  | { kind: 'success', results: HNResponse }
  | { kind: 'failure', error: string };

function display(action: Action) {
  if (action.kind === 'success') {
    console.log(action.results);
  }
}

function reducer(state: State, action: Action): State {
  switch (action.kind) {
    case 'request':
      return { isLoading: true };
    case 'success':
      return { isLoading: false, data: action.results };
    case 'failure':
      return { isLoading: false, error: action.error };
  }
}

function App() {
  const [query, setQuery] = useState<string>();
  const [{
    data,
    isLoading,
    error
  }, dispatch] = useReducer(reducer, { isLoading: false });

  useEffect(() => {
    let ignore = false;

    dispatch({ kind: 'request' });
    fetch(`https://hn.algolia.com/api/v1/search?query=${query}`)
        .then(
            (response: Response) => response.json().then(
                (results: HNResponse) => {
                    if (!ignore) dispatch({ kind: 'success', results });
                }
```

```
            ),
            (error: Error) => dispatch({ kind: 'failure', error: error.message })
        );


    return () => { ignore = true; }
  }, [query]);

  return (
    <div>
      <input
        value={query}
        onChange={e => setQuery(e.target.value)}
        placeholder="Type to search" />
      {isLoading && <span>Loading...</span>}
      {error && <span>Error: {error}</span>}
      <ul>
        {data && data.hits && data.hits.map(item => (
          <li key={item.objectID}>
            <a href={item.url}>{item.title}</a>
          </li>
        ))}
      </ul>
    </div>
  );
}

ReactDOM.render(
    <App />,
    document.getElementById("root")
);
```

Even better state representation

If we take a look at the interface representing the state of this component, we will notice that some combinations of properties are not valid.

For example, it is not possible that `isLoading === true` while `data` is not empty.

Similarly, `error` and `data` cannot be both defined at the same time.

How can we improve this? Let's represent the state as a state machine!

```
type State =
 | { status: 'empty' }
 | { status: 'loading' }
 | { status: 'error', error: string }
 | { status: 'success', data: HNResponse }
```

Why is this approach better? Because it makes illegal states unrepresentable. The previous interface definition allowed certain combinations of property

values even though we were sure that they would never occur in reality.

In a more complex component, this could force us to add some typecasts or handle impossible situations. Thanks to discriminated unions we can eliminate this issue.

It is generally a good idea to make your types match reality as closely as possible.

```
import * as React from "react";
import * as ReactDOM from "react-dom";
import { Component, useState, useEffect, useReducer } from 'react';

type HNResponse = {
  hits: {
    title: string;
    objectID: string;
    url: string;
  }[]
};

type State =
  | { status: 'empty' }
  | { status: 'loading' }
  | { status: 'error', error: string }
  | { status: 'success', data: HNResponse };

type Action =
  | { kind: 'request' }
  | { kind: 'success', results: HNResponse }
  | { kind: 'failure', error: string };

function display(action: Action) {
  if (action.kind === 'success') {
    console.log(action.results);
  }
}

function reducer(state: State, action: Action): State {
  switch (action.kind) {
    case 'request':
      return { status: 'loading' };
    case 'success':
      return { status: 'success', data: action.results };
    case 'failure':
      return { status: 'error', error: action.error };
  }
}

function App() {
  const [query, setQuery] = useState<string>();
  const [state, dispatch] = useReducer(reducer, { status: 'empty' });

  useEffect(() => {
    let ignore = false;
```

```
    dispatch({ kind: 'request' });
    fetch(`https://hn.algolia.com/api/v1/search?query=${query}`)
        .then(

            (response: Response) => response.json().then(
                (results: HNResponse) => {
                    if (!ignore) dispatch({ kind: 'success', results });
                }
            ),
            (error: Error) => dispatch({ kind: 'failure', error: error.message })
        );

    return () => { ignore = true; }
  }, [query]);

  return (
    <div>
      <input value={query} onChange={e => setQuery(e.target.value)} />
      {state.status === 'loading' && <span>Loading...</span>}
      {state.status === 'success' && <ul>
        {state.data && state.data.hits && state.data.hits.map(item => (
          <li key={item.objectID}>
            <a href={item.url}>{item.title}</a>
          </li>
        ))}
      </ul>}
      {state.status === 'error' && <span>Error: {state.error}</span>}
    </div>
  );
}

ReactDOM.render(
    <App />,
    document.getElementById("root")
);
```

The final lesson looks into another advanced typing scenario.

RxJS and TypeScript: `pipe`

This lesson looks into typing the `pipe` function in RxJS as an example of typing complex variadic functions.

WE'LL COVER THE FOLLOWING

- Overview
- What is piping?
- Typing `pipe`
- `pipe` in RxJS

Overview

If you're familiar with the RxJS library, you should be accustomed with the concept of **pipeable operators**. It's a mechanism of composing operators (applying an operator on top of other operators) that replaced chained method calls (`stream$.map(...).filter(...)`) to make the library tree-shakable. It's based on the idea of function composition from functional programming. Looking into how `pipe` is typed is a good exercise in advanced typing.

What is piping?

First, let's look at some examples of using `pipe`.

```
const number$ = of(1, 2, 3, 4, 5).pipe(
    filter(x => x % 2 === 0),
    map(x => x * x)
);

const response$ = fromEvent(button, 'click').pipe(
    debounceTime(500),
    tap(() => { console.log('fetching data...'); }),
    switchMap(()=> fetch(`http://example.com/api/status`)),
    switchMap(response => response.json()),
```

```
    catchError(() => empty())
);
```

As you can see, `pipe` can be called with a different number of arguments. What's more, these arguments can be very diverse as any RxJS operator, with any number of parameters, can be used. Finally, the type of each argument depends on the type of the previous argument. For example, the type of the function provided to the second `switchMap` is `(value: Response) => Promise<any>` because the function provided to the first `switchMap` returns a `Response`.

Typing `pipe`

Let's try to sketch the type of `pipe`. First of all, how do we deal with the fact that any number of arguments is provided? Although TypeScript allows us to type simple functions accepting any number of parameters (e.g. `(...numbers: number[]) => number`), this mechanism won't work in this case. The problem is that, as already mentioned, the type of each parameter is different and may depend on the type of the previous parameter. We'll deal with this problem by providing a number of function overloads, each with a different number of parameters.

Next, let's think about the types of parameters. Each parameter represents an operator. An RxJS operator is a function that takes a stream and returns a stream. The type of values in the returned stream might differ from the type of elements in the source stream. What's more, the type of elements in the source stream must be the same as the type of elements in the stream returned by the previous operator.

Given all these points, the types for `pipe` with up to three parameters might look like this:

```
function pipe<S, R>(
    operator1: (s1: Observable<S>) => Observable<R>,
): Observable<R>;
function pipe<S, T1, R>(
    operator1: (s1: Observable<S>) => Observable<T1>,
    operator2: (s2: Observable<T1>) => Observable<R>,
): Observable<R>;
function pipe<S, T1, T2, R>(
    operator1: (s1: Observable<S>) => Observable<T1>,
    operator2: (s2: Observable<T1>) => Observable<T2>,
```

```
    operator3: (s3: Observable<T2>) => Observable<R>,
): Observable<R>;

function pipe(...operators: ((s1: Observable<unknown>) => Observable<unkno
wn>)[]): Observable<unknown> { /* ... */ }
```

As you can see, function overloads in TypeScript are created by providing several function signatures and one implementation. When `pipe` is called, TypeScript will look for a matching overload and use its types. If it cannot find a matching overload, it will fall back to the type of implementation.

Each overload is a generic function with type arguments representing the type of elements in the source observable (`S`), element types of interim observables (`T1`, `T2`, ...), and the element type of the resulting observable (`T4`). Each argument is a function that takes an observable of one type and returns an observable of the "next" type.

`pipe` in RxJS

Let's now take a look at how `pipe` is implemented in RxJS. We'll look at the `pipe` *method* as opposed to the `pipe` *function*.

```
export declare class Observable<T> implements Subscribable<T> {
    pipe<A>(op1: OperatorFunction<T, A>): Observable<A>;
    pipe<A, B>(
        op1: OperatorFunction<T, A>,
        op2: OperatorFunction<A, B>,
    ): Observable<B>;
    pipe<A, B, C>(
        op1: OperatorFunction<T, A>,
        op2: OperatorFunction<A, B>,
        op3: OperatorFunction<B, C>,
    ): Observable<C>;
    pipe<A, B, C, D>(
        op1: OperatorFunction<T, A>,
        op2: OperatorFunction<A, B>,
        op3: OperatorFunction<B, C>,
        op4: OperatorFunction<C, D>,
    ): Observable<D>;
    /* ... */
}
```

As you can see, these definitions follow exactly the same pattern. The only difference is that instead of explicitly mentioning the function type, it uses an

difference is that instead of explicitly mentioning the function type, it uses an alias `OperatorFunction`, which makes the whole thing slightly more readable.

Conclusion

This lesson concludes the course.

Thank you so much for looking into this course! I hope you enjoyed it, and I'm confident you will find your newly acquired TypeScript skills helpful. The course focused on the most practical bits of advanced TypeScript knowledge. Every topic we've discussed should benefit your day-to-day productivity.

You've learned about the most important compiler flags responsible for increasing the strictness of type checking. The stricter the checks, the more potential errors you will find and fix before they reach production.

We've discussed generic types and functions in detail. You should now have no trouble using and creating your own generic types. You've also gained a new way of thinking about generic types as of type constructors.

Most importantly, you've learned how to compose types with discriminated unions. You're now able to express complex business rules in the type system and have the type checker ensure the correctness of your code.

You've seen various advanced type mechanisms in TypeScript and now have an intuition about when they can be useful. Finally, you've seen how to debug type errors and how to analyze complex types in popular libraries.